DAVE3610 Network and System Administration

Exam Notes – Theory

Written by Jørgen Skontorp

These notes are based on the lecture and course material from Ismail Hassand and Nakarin Phooripoom. As well as the book Linux System Administration by Paul Cobbaut.

I have used my custom GPT, *SysadminGPT* to help me create these notes, due to time constraints. SysadminGPT is fed with the syllabus for this course.

Licensed under the CreativeCommons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0) license.

Disclaimer: I do not take an responsibility for anything that may be wrong in these notes. Use at own risk.

1. Introduction to Network and System Administration....4
What is a System Administrator?....4
Responsibilities and Scope of Work....4
Evolution of the Sysadmin Role....4
Why do we need sysadmins?....5
Core sysadmin skills....5
Typical sysadmin tasks....5
Why Linux?....5
2. Operating Systems (Disks, partitions, and file systems)....6
Linux installation....6
Partitioning....6
Filesystems....10
Mounting....12
3. Managing Local Storage (RAID and LVM)....13
Storage types....13
Disk Interfaces....14
Logical Volume Management (LVM)....14
Redundant Array of Independent Disks (RAID)....16
4. Software Installation....18
Package....18
Package management tools....18
Software Installation Types....19
Handling Dependencies and Software Integrity....19
Security and Integrity in Package Management....19
5. Local Users Account Management....20
Access to a Linux Machine....20
Linux Login Process....20
Users and Their Properties....20
Group Affiliation....20
Management....21
6. Centralized User Account Management (LDAP, PAM and Kerberos)....22
The Challenge with Local Accounts....22
Typical Account Scenario....22
Centralized Authentication with LDAP....22
Linux Authentication Mechanisms (PAM and NSS)....23
Integration with Kerberos for Enhanced Security....24
Centralized Authentication Alternatives....25
Overview of LDAP, PAM, and Kerberos....25
How LDAP, PAM, and Kerberos Work Together....26
7. Manage Permission, Ownership and Resources....27
Authentication and Authorization....27
Access Control....27
Permissions....27
Inode....28
Special File Permissions (SUID, SGID, Sticky bit)....29
Commands for Managing Permissions....31
Access Control Lists (ACLs)....31
Issues with Discretionary Access Control (DAC)....31
Mandatory Access Control (MAC)....31
Default Permissions and Umask....32

Resource Management....32
Administration with sudo....32
8. Network Attached Storage (NAS, NFS, Kerberos)....34
NFS (Network File System)....34
NFS Security Problems....35
Kerberos with NFS....36
Components of NFS....37
Server Configuration....38
Client Setup....39
Dynamically Mounting With autofs....39
9. Storage Area Network (iSCSI)....41
Scenario and Problems with DAS....41
Solutions and Limitations....41
Transition to Network Storage....41
SAN Configuration and Technologies....42
iSCSI Details....42
Redundancy and Performance Enhancements....42
10.1. Dynamic Host Configuration Protocol (DHCP) and Static IP....43
Static IP Configuration....43
Challenges with Static IP Configuration....43
Dynamic Host Configuration Protocol (DHCP)....43
How DHCP Works....44
DHCP Messages Detailed....44
ISC DHCP Implementation....44
10.2. Domain Name Server (DNS)....45
Challenges and Solutions....45
DNS Configuration Files....45
How DNS Works....45
DNS Queries....46
DNS Server and Records....46
DNS Tools....46
11. Email System (SMTP, IMAP, Webmail)....47
Email System Components....47
Email Transmission: SMTP....47
Email Retrieval: IMAP and POP3....47
Email Storage: Mbox vs Maildir....47
Handling Multiple Domains....47
Security and Management....48
Webmail Systems....48
12. Miscellanous....49
Default ports....49
Useful commands....52
Configuration Files and Locations....55

1. Introduction to Network and System Administration

Network and System Administration involves the comprehensive management of computer networks and systems to ensure they operate effectively and meet the requirements of users and organizations. System administrators, or "sysadmins," play a crucial role in the daily operations of modern digital infrastructures.

What is a System Administrator?

A system administrator (sysadmin) is responsible for the maintenance, configuration, and reliable operation of computer systems, especially multi-user computers, such as servers. The sysadmin ensures that the system hardware, software, and related procedures adhere to organizational values, enabling staff, volunteers, and end users to carry out their tasks efficiently and effectively.

According to Mark Burgess, network and system administration involves assembling a networked system of computers and software—from workstations to supercomputers—and ensuring their continual operation. This includes addressing the challenges posed by users and external factors that may disrupt system functionality.

Responsibilities and Scope of Work

Sysadmins are tasked with a variety of responsibilities to ensure the smooth operation of network systems, which include:

- **Uptime and Reliability**: Ensuring systems are consistently available and perform at expected levels.
- **Performance Management**: Monitoring system performance and making adjustments to hardware or software configurations as needed.
- **Resource Management**: Allocating system resources effectively to meet both user needs and organizational budget constraints.
- **Security**: Safeguarding systems against threats, ensuring data integrity and access controls are in place and effective.

Evolution of the Sysadmin Role

The role of the sysadmin has evolved significantly with advancements in technology:

- **Integration**: Modern sysadmins must integrate old and new technologies seamlessly and maintain the infrastructure to support rapid organizational and technological changes.
- **DevOps**: The sysadmin role Linux is often the operating system of choice for sysadmins due to its robustness, security, and stability. It supports a wide range of hardware and networking setups and offers extensive customization options:
- **Open Source**: Linux's open-source nature allows sysadmins to modify the system to meet specific needs.
- **Community and Support**: A vast community and a plethora of distributions offer support for various types of environments, from small businesses to large enterprises.

- **Cost-Effectiveness**: Linux reduces operating costs, as most distributions are free with extensive free software available for various tasks.
- has become more development-oriented, emphasizing continuous development, integration, and deployment practices.
- **Automation**: A key focus in system administration now is to automate repetitive tasks to improve efficiency and reduce the possibility of human error.

Why do we need sysadmins?

Sysadmins are essential for maintaining the critical digital services that society relies upon, such as:

- **Critical Services**: Internet banking, public services, e-commerce, and social networks rely on continuous and uninterrupted service.
- **Reliability and Availability**: Businesses and customers demand services that are available around the clock, every day of the year.
- **Handling Complexity**: Sysadmins manage complex systems and dependencies, ensuring stability even when unexpected events occur.

Core sysadmin skills

A proficient sysadmin typically possesses skills across various domains, including, but not limited to: Networking, Operating Systems, Scripting and Automation, Security Protocols, Disaster Recovery and Backup Procedures

Typical sysadmin tasks

Day-to-day tasks of a sysadmin may include:

- Installing and configuring software and hardware.
- Managing network servers and technology tools.
- Setting up accounts and workstations.
- Monitoring performance and maintaining systems according to requirements.
- Ensuring security through access controls, backups, and firewalls.

Why Linux?

Linux is often the operating system of choice for sysadmins due to its robustness, security, and stability. It supports a wide range of hardware and networking setups and offers extensive customization options:

- **Open Source**: Linux's open-source nature allows sysadmins to modify the system to meet specific needs.
- **Community and Support**: A vast community and a plethora of distributions offer support for various types of environments, from small businesses to large enterprises.
- **Cost-Effectiveness**: Linux reduces operating costs, as most distributions are free with extensive free software available for various tasks.

2. Operating Systems (Disks, partitions, and file systems)

Linux installation

- **Preparation**:
  - **ISO Image**: Always ensure you download the ISO image from a reliable and official source to avoid corrupted files and security risks. Examples are directly from the distribution's website or trusted mirrors.
  - **Media**: Depending on the system requirements and available resources, choose whether to burn the ISO to a CD, DVD, or create a bootable USB drive. Tools like `dd` for USB or `Brasero` for DVDs are commonly used in Linux environments.
- **Boot Options**:
  - **BIOS/UEFI Settings**: Ensure the system's BIOS or UEFI is set up to boot from the correct device. This might require changing the boot order to prioritize USB or DVD drives.
  - **Network Boot**: For enterprise environments, consider setting up a PXE (Preboot eXecution Environment) server for network installations, which can greatly simplify the deployment of Linux across multiple machines.
- **Installation Process**:
  - **Basic Configuration**: Follow the on-screen prompts to configure language, keyboard layout, and timezone.
  - **Network and Hostname**: Setting up the network and hostname correctly during installation ensures that the system can communicate on the network and be identified properly.
  - **User Accounts**: Securely set up a root account and at least one regular user account. Consider setting strong passwords or setting up SSH key-based authentication for additional security.

Partitioning

Partitioning means splitting the hard drive into isolated parts. Each partition will behave as a separate hard drive. Before installing Linux on a machine, the hard disk must be partitioned. To make this process easier during installation, many distributions offer an option of using the entire hard disk or the remaining space on the disk.

Goals of Partitioning

- **Organization and Management**: Separating system files from user files (e.g., `/home`, `/var`) can prevent system issues if user files consume too much disk space.
- **Security and Stability**: Different partitions can be mounted with different options to enhance security (e.g., `noexec` on `/tmp` to prevent execution of binaries).

- **Performance**: On rotational disks, placing frequently accessed data on outer tracks can improve access speed. Using separate partitions can aid in optimizing the placement of data.

Types of Partitions

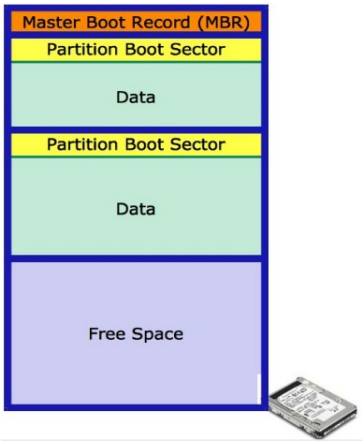
- **Primary and Extended**: Most traditional BIOS systems use MBR, which supports up to four primary partitions, or three primary and one extended partition. The extended partition can contain multiple logical partitions.
- **GPT (GUID Partition Table)**: Used by newer UEFI systems, supports a much larger number of partitions (up to 128 in Windows, more in Linux), and larger disks.

File Systems

- **Choosing a File System**: Common choices include Ext4, XFS, and Btrfs for Linux systems. Consider the file system features such as journaling, support for large files, and data integrity mechanisms.
- **Formatting**: After creating partitions, they must be formatted with a file system before use. This is usually done during the installation process.

Advanced Considerations

- **Swap Space**: Allocating swap space is crucial for system performance, especially on systems with limited RAM.
- **Mount Points**: Assigning mount points during installation helps determine where each partition will be accessible in the file system hierarchy.
- **Automated Partitioning**: Some installers offer automated partitioning choices, which are useful for beginners or standard installations, but manual partitioning is preferred for customized setups.



Before we have done any partitions on a new disk, there is “just” free space. The disk has already been formatted at a low level from the factory. There are no partitions nor filesystem.

This diagram represents the typical structure of a hard disk using the Master Boot Record (MBR) partitioning scheme. The visualization highlights the organization of data and system information on a hard disk that has been partitioned into two sections. The MBR sits at the very beginning of the disk and contains the bootloader. The Partition Boot Sector is at the beginning of each partition. It contains necessary code to initiate the booting process. The data areas are regions that store actual user data and system files. The free space is unallocated sections of the disk that follows the existing partitions.

Harddisk

Understanding Data Storage on Hard Disks

- **Data Blocks**: Data on hard disks is organized into blocks, which are the smallest unit of data storage. A block typically contains 512 bytes, which has been the standard for many years.
- **Advanced Format Disks**:
    - With technological advancements, many newer hard disks use an "advanced format" where the block size is increased to 4KB (4096 bytes).
    - This increase in block size allows for more efficient use of disk space and improved error correction capabilities.

Logical Block Addressing (LBA):

- **Function**: LBA is a method used to specify the location of these blocks of data. Unlike older methods that used cylinder-head-sector (CHS) addressing, LBA provides a linear and straightforward way to address blocks.
- **Advantages**: LBA simplifies the interface between the operating system and the disk hardware. Instead of dealing with physical disk geometries, the system can treat the storage as a sequential list of blocks (LBA 0, 1, 2, ...).
- **Usage**: LBA makes it easier for operating systems to manage and access data on larger storage devices, as it abstracts the physical details of disk storage into a simple, linear model.

Physical Structure of Hard Disks:

- **Components**:
    - **Platters**: Disks are composed of one or more platters, which are the circular disks that store data.
    - **Tracks**: Each platter contains tracks, which are concentric circles around the disk.
    - **Sectors**: Tracks are further divided into sectors, which are the smallest physical storage units on the disk.
- **Relation to LBA**: While the physical structure consists of platters, tracks, and sectors, LBA provides a simplified way of addressing these sectors, making it easier for software to read and write data without concerning itself with the disk’s physical layout.

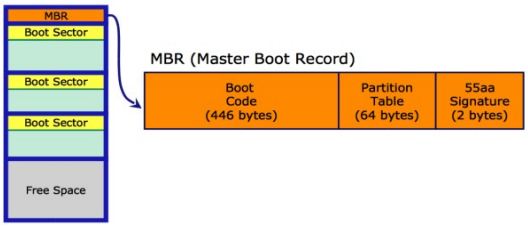
Implications for System Boot Process:

- During the boot process, the system BIOS or UEFI firmware uses LBA to locate the Master Boot Record (MBR) or EFI System Partition on the disk. This is critical for loading the bootloader and initiating the operating system startup.
- The simplicity of LBA aids in the compatibility and flexibility of boot processes across different systems and storage technologies.

Basic Input/Output System (BIOS)

- **Function**: The BIOS is firmware embedded on the motherboard, responsible for initializing hardware and performing system checks right after the computer is turned on.
- **Boot Process**: Once the initial hardware checks are complete, the BIOS identifies the bootable device and proceeds to load the boot loader from this device.

Master Boot Records (MBR)



Location and Structure:

- The Master Boot Record is located at the very first sector of the hard drive. This sector is crucial because it contains the boot code and the partition table.
- **Size**: The MBR occupies 512 bytes, which include the boot code, the partition table for up to four primary partitions, and a signature at the end to validate its authenticity.

- **Boot Code**:
  - This segment of the MBR contains executable code that is primarily responsible for initiating the operating system's boot sequence. This code can be part of various boot loaders like GRUB or LILO, depending on the installed operating system.
- **Partition Table**:
  - Holds details about the size and location of each partition. ***The MBR supports up to four primary partitions directly***, which can be a limitation for complex system configurations.

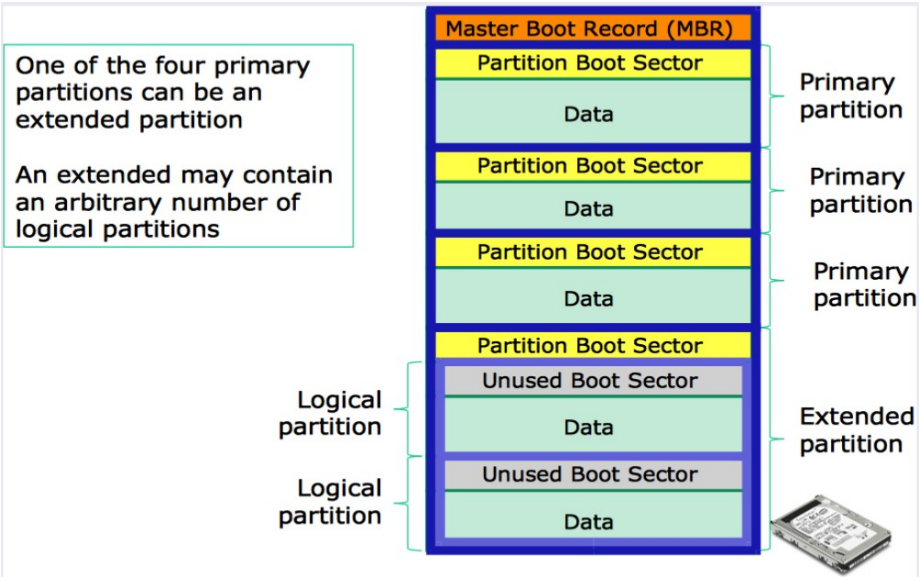
Globally Unique Identifiers Partition Table (GPT)

- **Advancement Over MBR**:
  - As an advancement over the older MBR system, GPT offers a more robust mechanism for partitioning disks. It is associated with the Unified Extensible Firmware Interface (UEFI), which is designed to replace the legacy BIOS system.
- **Features**:
  - **Capacity**: Unlike MBR, GPT does not limit the number of partitions to four primary partitions. It supports up to 128 partitions by default on most systems, removing the need for extended and logical partitions.
  - **Robustness**: GPT includes redundancy and checksums to improve the integrity and error resistance of the partition table.
- **UEFI and GPT**:
  - UEFI and GPT are parts of modern systems aimed at overcoming the limitations of BIOS and MBR, respectively. UEFI, for instance, supports larger disk sizes and faster boot times, while GPT allows for a greater number and more flexible arrangement of disk partitions.

Extended and logical partitions

This concept extends the capacity of an MBR-partitioned disk beyond the four-primary partition limit by introducing an extended partition. The extended partition can house multiple logical partitions:

- **Extended Partition**: Unlike primary partitions, an extended partition does not directly store data or host operating systems. Instead, it serves as a container that can be subdivided into smaller, logical partitions.
- **Logical Partitions**: These partitions reside within the extended partition. They are often used to separate user data from operating system files or to organize data more efficiently. Logical partitions are treated similarly to primary partitions by the operating system but are nested within the extended partition.
- **Practical Use**: This setup is particularly useful for users who need to create more than four partitions on a disk. It offers flexibility and better space management without the need to add additional physical disks.



Filesystems

Purpose of Filesystems:

- **Filesystem Role**: Before operating systems and applications can utilize the disk space, a filesystem must be created on a partition. The filesystem organizes and manages files and directories on your partitions and efficiently keeps track of where everything is stored.
- **Sector Size**: If not specified, the sector size of a filesystem defaults to 512 bytes, although newer hard drives and certain filesystems might use larger sizes like 4096 bytes (4 KB) for improved efficiency and error correction.

Types of Linux Filesystems:

- **Ext Family (Ext2, Ext3, Ext4)**:
  - **Ext2**: Non-journalling filesystem, primarily used in scenarios where journaling is not required or desirable (e.g., simple storage devices or flash drives).
  - **Ext3**: Introduced journaling on top of Ext2, which helps in improving reliability and recovery speed after crashes.
  - **Ext4**: Current default for many Linux distributions; supports larger filesystems and volumes, more efficient allocation of disk space, and increased performance.
- **Other Filesystems**:
  - **XFS**: Known for its high performance and scalability, particularly on large filesystems.
  - **Btrfs**: Offers advanced features like snapshotting, dynamic inode allocation, and integrated device management.
  - **ZFS**: Not traditionally included with Linux due to licensing issues but known for its robustness and advanced features such as pooling, snapshots, and checksums.
  - **ReiserFS and JFS**: Less commonly used today but notable for specific use cases in the past.

Creating a Filesystem:

- **Command**: Typically, a filesystem is created using the `mkfs` command followed by a type specifier, such as `mkfs.ext4` for creating an Ext4 filesystem.
- **Example Command**: `mkfs -t ext4 /dev/sda1` would format the partition `/dev/sda1` with the Ext4 filesystem.

Filesystem Features:

- **Superblock**: Stores crucial information about the filesystem, such as its size, block size, empty and filled blocks, and filesystem mount points.
- **Inode Table**: Lists inodes, which contain metadata about files but do not contain actual file data. Metadata includes permissions, owner, size, and pointers to data blocks.
- **Data Blocks**: The actual locations where file and directory content is stored on the disk.

Filetypes

File Types in Linux:

- **Directories (`d`)**:
  - **Characteristics**: Folders that contain files and other directories.
  - **Command to Create**: `mkdir directory_name`
- **Regular Files (`-`)**:
  - **Characteristics**: Contains data, text, or program information.
  - **Command to Create**: `touch filename`
- **Symbolic Links (`l`)**:

    - **Characteristics**: A type of file that is a reference or pointer to another file or directory. It allows multiple access paths for a single file.
    - **Command to Create**: `ln -s target link_name`
- **Special Files**:
    - **Characterized as**: Used mainly for device input and output. Linux treats devices like files, which simplifies device management.
    - **Two types**:
        - **Character Devices (`c`)**: Unbuffered devices that handle data in a character-by-character fashion.
        - **Block Devices (`b`)**: Buffered devices that handle data in blocks.
    - **Command to Create**: `mknod device_name [b|c] major minor`
        - **`mknod`**: Creates device files given a type (block `b` or character `c`), a major number, and a minor number, which are specific to the device driver in the kernel.

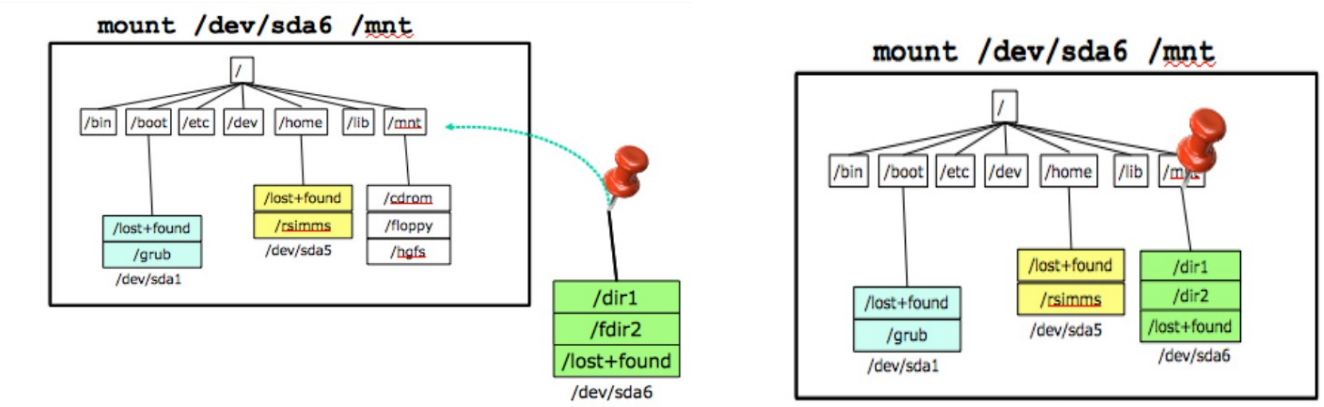
Mounting

Fundamentals of Mounting:

- **Purpose**: Before the operating system can use any file system on a disk, it must be mounted. Mounting a file system attaches it to a specific directory in the existing directory structure, making its contents accessible.
- **Root File System**: The root file system (`/`) must be available and mounted at boot time since it contains critical system files and directories.

Mounting Process:

- **Temporary Mounting**: File systems can be temporarily mounted using the `mount` command, which attaches the file system to a specified mount point (directory). For example, `mount /dev/sda6 /mnt` would mount the file system located on `/dev/sda6` to the directory `/mnt`.
- **Permanent Mounting**: To ensure a file system remains mounted after a reboot, an entry must be added to the `/etc/fstab` file.



3. Managing Local Storage (RAID and LVM)

Storage types

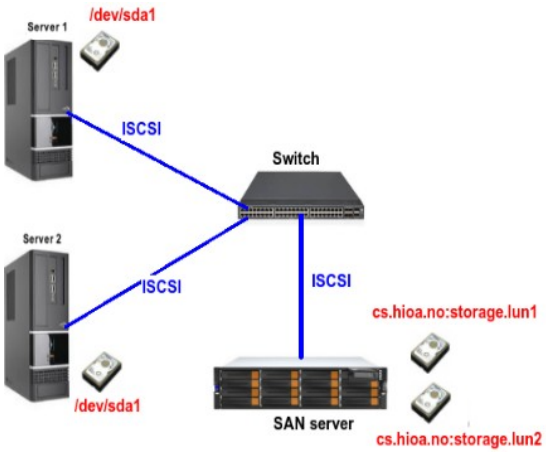
Direct Attached Storage (DAS):

Definition: DAS is digital storage directly attached to the computer accessing it, as opposed to being networked. Examples include internal hard drives, external hard drives, and storage on external drives connected via USB or eSATA.

Usage Scenario: Ideal for individual users needing fast, dedicated storage on their computer or server without the necessity for network sharing.

Storage Area Network (SAN):

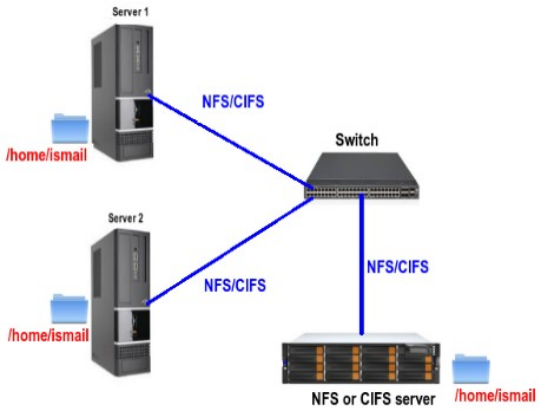
Definition: A SAN is a network of storage devices that can be accessed by multiple computers, allowing stored data to be accessed by various client systems. SANs are highly efficient and versatile, typically used in enterprise environments where large volumes of data are stored and accessed across networks.



Usage Scenario: Perfect for complex storage needs where data needs to be centralized yet highly accessible, such as in large databases and transactional systems.

Network Attached Storage (NAS):

Definition: NAS is a dedicated file storage device that provides local area network (LAN) users with centralized, consolidated disk storage through a standard Ethernet connection.



Usage Scenario: Suited for small to medium-sized organizations needing easy, consolidated data access for multiple clients at a lower price point than SAN.

Disk Interfaces

A hard drive connects to the computer via a specific type of a disk interface The interface on the hard drive must match the corresponding interface on the motherboard

Internal Interfaces:

- **IDE/PATA (Parallel ATA)**: An older disk interface standard used primarily for connecting storage devices in computers.
- **SATA (Serial ATA)**: The common standard for connecting and transferring data from hard drives to the computer's motherboard.
- **SCSI (Small Computer System Interface)**: Used in both server and workstation markets for connecting external devices like hard drives, scanners, etc.
- **SAS (Serial Attached SCSI)**: A point-to-point serial protocol that moves data to and from computer-storage devices such as hard drives and tape drives.
- **SSD (Solid State Drive)**: Uses integrated circuit assemblies to store data persistently, typically using flash memory, and functioning as secondary storage in the hierarchy of computer storage.

External Interfaces:

- **USB (Universal Serial Bus)**: An industry standard for cables, connectors, and protocols for connection, communication, and power supply between computers and devices.
- **eSATA (External SATA)**: A variant of SATA meant for external connectivity, offering robust and high-speed connections for external storage devices.
- **Firewire (IEEE 1394)**: A serial bus interface standard for high-speed communications and isochronous real-time data transfer, commonly used in video industries.

Logical Volume Management (LVM)

What is Logical Volume Management (LVM)?

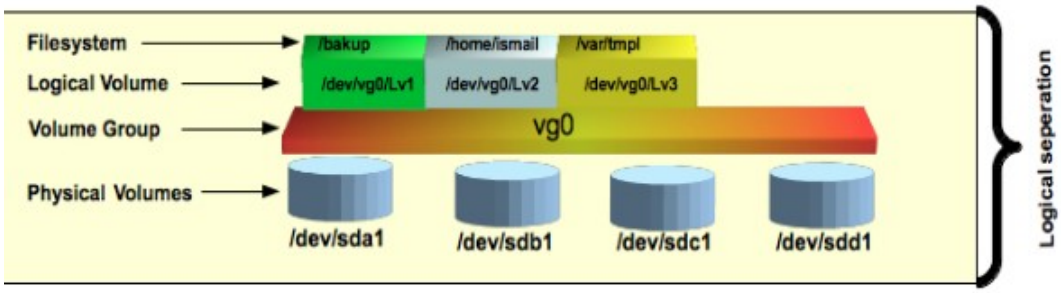
Logical Volume Management (LVM) is an advanced storage management technology that provides administrators with much greater flexibility compared to traditional disk partitioning. LVM allows for the dynamic resizing of disk partitions and the ability to span a file system across multiple physical disks.

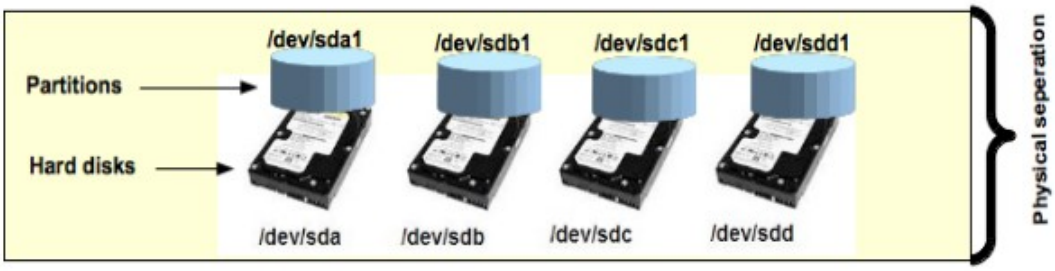
Key Advantages:

- **Flexibility**: Volumes can be resized and moved freely across the available physical disks without system disruption.
- **Ease of Management**: Simplifies storage management by abstracting the details of the underlying physical disks.
- **Storage Pooling**: Multiple physical disks can be grouped into a single logical volume, making better use of available storage and simplifying file system management.

The contrast between traditional partitioning and LVM can be summarized as:

- **Traditional Partitioning**: Once set, partitions are fixed in size and location. Resizing or moving partitions often requires unmounting the partition and potential data loss if not done carefully.
- **LVM**: Provides a layer of abstraction above the physical disks, allowing partitions (logical volumes) to be resized and moved without impacting data.





Step-by-Step LVM Setup Process:

1. **Initialize Physical Volumes**:
   - Command: `pvcreate /dev/sda1 /dev/sdb1 /dev/sdc1 /dev/sdd1`
   - Purpose: Initializes physical disks or partitions as physical volumes that can be used in LVM.
2. **Create a Volume Group**:
   - Command: `vgcreate vg0 /dev/sda1 /dev/sdb1 /dev/sdc1 /dev/sdd1`
   - Purpose: Combines multiple physical volumes into a volume group, providing a pool of disk space from which logical volumes can be allocated.
3. **Expand Volume Group**:
   - Command: `vgextend vg0 /dev/sde1`
   - Purpose: Adds additional physical volumes to an existing volume group to increase its capacity.
4. **Create Logical Volumes**:
   - Command: `lvcreate -L 100G vg0 -n lv1`

    - Purpose: Allocates space from the volume group to create a logical volume of a specified size (100 GB in this case).
5. **Format Logical Volume with File System**:
    - Command: `mkfs -t ext4 /dev/vg0/lv1`
    - Purpose: Creates a file system (ext4) on the logical volume, making it ready to mount and use.

Visualization and Management Tools:

- **Commands like `lvdisplay`, `vgdisplay`, and `pvdisplay`** provide detailed information about logical volumes, volume groups, and physical volumes, respectively.

Practical Considerations

- **Dynamic Resizing**: Logical volumes can be resized dynamically, supporting system growth and changing storage requirements.
- **Data Safety**: While LVM offers flexibility, it is still crucial to maintain regular backups as data across multiple physical disks can increase the risk of data loss if not managed carefully.
- **Performance**: Managing volumes across multiple disks can introduce complexity. However, LVM is designed to minimize performance overhead while providing substantial management benefits.

Redundant Array of Independent Disks (RAID)

RAID (Redundant Array of Independent Disks) is a data storage virtualization technology that combines multiple physical disk drive components into a single logical unit. The purpose of RAID is to enhance data redundancy and increase the performance of disk storage. The operating system sees RAID as a single logical disk.

Objectives of RAID:

- **Improved I/O Performance**: By distributing the data across multiple disks, RAID can significantly boost the speed of data reading and writing.
- **Increased Redundancy**: RAID configurations can protect data against the failure of one or more disks, depending on the RAID level used.

Common RAID Levels

1. **RAID 0 (Striping)**:
    - Distributes data across multiple disks to maximize read/write performance.
    - **No redundancy**: The failure of a single disk results in total data loss.
    - **Minimum Disks**: 2
2. **RAID 1 (Mirroring)**:
    - Creates an exact copy of data on two or more disks.

    - **Redundancy**: Provides 100% redundancy. If one disk fails, data is still available from other disks.
    - **Performance**: Read performance is enhanced (up to double), while write performance can be slightly lower due to the duplication of data.
3. **RAID 5 (Striping with Parity)**:
    - Data and parity (error checking) information are striped across three or more disks.
    - **Redundancy**: Can withstand the failure of one disk without losing data.
    - **Performance**: Offers good read speeds but write performance can be affected due to parity calculations.
    - **Minimum Disks**: 3
4. **RAID 6 (Dual Parity)**:
    - Similar to RAID 5 but with an additional parity block; data and two parity blocks are striped across four or more disks.
    - **Redundancy**: Can handle the simultaneous failure of two disks.
    - **Performance**: Provides high data protection and good read performance but has slower write performance due to dual parity calculations.
    - **Minimum Disks**: 4

Hardware RAID vs. Software RAID

- **Hardware RAID**:
    - **Pros**: Generally offers better performance as it uses dedicated processors to handle RAID processing.
    - **Cons**: More expensive due to the need for additional hardware (RAID controllers).
- **Software RAID**:
    - **Pros**: Less costly as it uses the host system's CPU for RAID functionality. With modern CPUs, software RAID can offer performance that rivals hardware RAID for some levels.
    - **Cons**: Performance might still be less optimal than hardware RAID, particularly for high data throughput and fault tolerance features.

Linux and RAID

- **Implementation**: Linux supports software RAID through various tools such as `mdadm`.
- **Performance**: The software RAID implementation in Linux is considered robust and efficient, suitable for many applications where hardware RAID might not be affordable.

Understanding RAID levels and choosing the right one are crucial for system administrators tasked with designing and maintaining resilient and high-performance storage solutions. RAID technology not only enhances storage capabilities but also significantly increases data reliability and availability, which are essential for critical business operations.

4. Software Installation

In Linux systems, software installation is a fundamental task that every system administrator must master. This involves not only the ability to install and update software but also the knowledge of different tools and methods provided by the operating system for effective software management.

Package

A **package** in Linux is a compressed file archive containing all the files necessary for the application to run. This includes the executable files, configuration files, documentation, and libraries. Each package contains metadata which describes the package's contents and includes information like dependencies, version, and a description.

Package Components:

- **Files**: Executables, configuration files, and libraries.
- **Metadata**: Descriptions, pre/post installation scripts, and property information.
- **Dependencies**: Other packages required for the package to function properly.

Package management tools

Linux uses a variety of tools for package management, categorized into low-level and high-level tools.

Low-Level Tools:

- **dpkg** (Debian-based systems): Installs, removes, and builds packages but does not handle dependencies.
    - **Commands**:
        - `dpkg -i package-name.deb`: Install/update a package.
        - `dpkg -r package-name`: Remove a package (keeps configuration files).
        - `dpkg --purge package-name`: Remove a package including configuration files.

High-Level Tools:

- **APT (Advanced Package Tools)**: Automates the process of installing, upgrading, and removing software and is capable of resolving dependencies.
    - **Commands**:
        - `apt install package-name`: Install new packages.
        - `apt remove package-name`: Remove packages but keep configuration files.
        - `apt --purge remove package-name`: Remove packages including configuration files.
- **APT Utilities**:
    - `apt-cache search package-name`: Search for packages.
    - `apt-get`, `aptitude`: Additional tools for handling packages efficiently.

Software Installation Types

1. **Using Package Managers**:
    - **Advantages**: Handles dependencies, easy to update and manage.
    - **Method**: Using `apt`, `yum`, or other package managers to install from repositories.
2. **Compiling from Source**:
    - Often necessary when specific configurations are required or when the latest software versions are not available as packages.
    - **Steps**:
        - **Configure**: Prepares the software to be compiled on your specific system.
        - **Compile**: Translates source code into executable code.
        - **Install**: Installs the software onto the system.
    - **Commands**:
        - `./configure`: Configures the software.
        - `make`: Compiles the software.
        - `make install`: Installs the software.

Handling Dependencies and Software Integrity

- **Checkinstall**: Instead of using `make install`, `checkinstall` can be used to create a Debian package and install it, which helps in easy uninstallation and maintains the package database.
- **Alien**: Converts packages between different Linux distributions, e.g., RPM to Debian.

Security and Integrity in Package Management

- Ensuring the integrity and security of software is critical. This includes:
    - **Checksums**: Verifying the integrity of downloaded packages.
    - **Signatures**: Ensuring packages are from a trusted source and haven't been tampered with.
    - **APT Security**: Uses standard public keys to verify signed packages, ensuring they are secure before installation.

5. Local Users Account Management

Managing user accounts is a fundamental responsibility of system administrators, ensuring appropriate access control and security measures are in place. This lecture covers various aspects of user account management on Linux systems, including user properties, user and group management commands, and user authentication processes.

Access to a Linux Machine

Linux utilizes a user and group-based system to manage permissions and access. User accounts can be organized in two primary ways:

1. **Locally**: Users and groups are defined and managed on each individual machine.
2. **Centrally**: Using one or more authentication servers to manage user accounts across multiple systems, commonly implemented using LDAP or Kerberos.

Linux Login Process

The login process in Linux involves several checks:

- Verification of the user and group existence.
- User permission to log in from the specified location.
- Correctness of the user's password with a limit on the number of retry attempts.
- Expiry check on the user's password, prompting for a change if necessary.
- Setting environment variables like the home directory.
- Starting the user's shell, leading to a command line or graphical interface.

Users and Their Properties

User properties in Linux are stored in `/etc/passwd` for general user information and `/etc/shadow` for secured password data. Key properties when creating a user include:

- **Username**: Unique identifier for user login.
- **Password**: Secured entry for accessing the user account.
- **User ID (UID)**: Numeric identifier assigned to each user.
- **Primary Group ID**: Default group assigned to the user.
- **GECOS**: General information field, optionally used for the user's full name or contact details.
- **Home Directory**: Personal storage directory for user files.
- **Default Shell**: Command interpreter assigned to the user.

Group Affiliation

- Each user must belong to at least one primary group, with the possibility of belonging to additional secondary groups.
- Primary group information is stored in `/etc/passwd`, while secondary group memberships are listed in `/etc/group`.

Management

Managing User Accounts

User management involves several key commands:

- **`useradd`**: Create a new user with specified properties such as username, UID, home directory, and default shell.
- **`usermod`**: Modify existing user properties.
- **`userdel`**: Remove a user account, optionally including the user's home directory with `-r`.

Managing Group Accounts

- **`groupadd`**: Add a new group to the system.
- **`groupmod`**: Modify properties of an existing group.
- **`groupdel`**: Delete a group.

Password Management

- **`passwd`**: Assign or change user passwords.
- **`chage`**: Modify user password expiry information.

Advanced User Management Tools

- **`newusers`**: Batch create or update users from a text file.
- **`chpasswd`**: Update passwords for multiple users in a batch mode.
- **`pwgen`**: Generate random passwords for users.

Querying User and Group Information

- **`id`**: Display user UID, GID, and group memberships.
- **`getent`**: Fetch entries from databases supported by the Name Service Switch libraries, such as `/etc/passwd` or LDAP.

Demonstrations and Practical Commands

- **`sudo apt install pwgen`**: Demonstrate installing and using `pwgen` to generate passwords.
- **`useradd`, `usermod`, `userdel`**: Demonstrate how to manage user accounts on a Linux system.
- **`chpasswd`**, **`newusers`**: Show how to efficiently manage passwords and user accounts in batch mode.

Configuration Files

- **`/etc/default/useradd`**: Sets default values for new users.
- **`/etc/login.defs`**: Specifies the default environment and login settings for users

6. Centralized User Account Management (LDAP, PAM and Kerberos)

In the realm of networked environments, managing local user accounts on individual machines becomes untenably complex as organizations expand. The traditional approach, where each machine maintains its own set of user management files like `/etc/passwd`, `/etc/shadow`, and `/etc/group`, introduces inefficiencies and security vulnerabilities, particularly when users require access across multiple systems. To streamline user management and enhance security, organizations increasingly turn to centralized authentication solutions such as LDAP (Lightweight Directory Access Protocol), which centralizes user credentials in a read-optimized, accessible database. Alongside LDAP, technologies like Pluggable Authentication Modules (PAM) and Kerberos fortify authentication by separating authentication mechanisms from application logic and securing credentials with strong cryptography, respectively. These integrations provide robust frameworks that support scalable, secure, and efficient user management across complex IT infrastructures.

The Challenge with Local Accounts

Local Account Management Issues:

- Initially, managing a single machine with local accounts is straightforward. Each user added requires updating local files: `/etc/passwd`, `/etc/shadow`, and `/etc/group`.
- As more machines are added, consistency becomes challenging. User identifiers such as UID and GID may differ across systems, complicating user management and leading to issues with permissions, especially when sharing resources like networked file systems.
- Managing passwords and home directories across multiple machines increases complexity and introduces security risks.

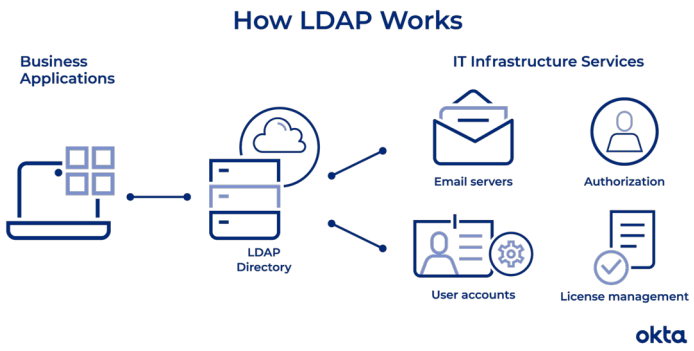
Typical Account Scenario

- **Example:** A student at OsloMet requires access to various IT services, each potentially requiring separate account creation and management. This situation underscores the inefficiency and risk of duplicating account management efforts across multiple systems.

Centralized Authentication with LDAP

LDAP (Lightweight Directory Access Protocol):

- LDAP is designed for more frequent read/search operations than updates/writes, making it ideal for managing user information across multiple systems.



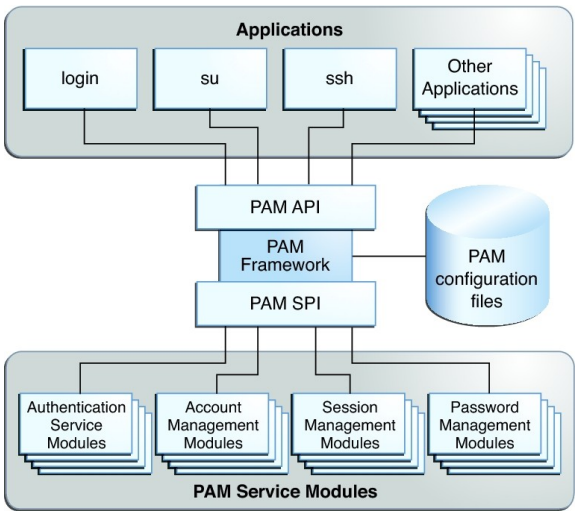
- It structures data in a hierarchical manner, often reflecting geographic or organizational boundaries, allowing for efficient data management and query.

Common LDAP Implementations:

- Microsoft Active Directory
- OpenLDAP
- 389 Directory Server (Red Hat)
- Novell eDirectory
- IBM Tivoli Directory Server

Linux Authentication Mechanisms (PAM and NSS)

PAM (Pluggable Authentication Modules):



- Provides a framework that decouples actual authentication from application logic. Applications interact with the PAM stack to perform authentication-related activities without needing to handle specifics.
- **PAM Modules:** Classified into types for different functions:
    - Authentication modules: Verify identity.
    - Account management modules: Handle account and session permissions.
    - Session management modules: Manage actions at the start/end of sessions.
    - Password management modules: Deal with password changes.
- **PAM Control Flags:** Handles the success or failure of a module during the authentication process.
    - **requisite**:
        - This flag is strict. If a module flagged as `requisite` fails, PAM immediately stops processing and returns a failure response to the service. No further modules are processed after a requisite module fails, making it essential for enforcing critical security policies.
    - **required**:
        - Modules marked as `required` must succeed for authentication to proceed. However, unlike `requisite`, if a `required` module fails, PAM continues to process the remaining modules in the stack to collect all failure data, but will ultimately deny the service at the end.
    - **sufficient**:
        - A `sufficient` module can bypass the rest of the authentication process if it succeeds, as long as no previous `required` modules have failed. If a

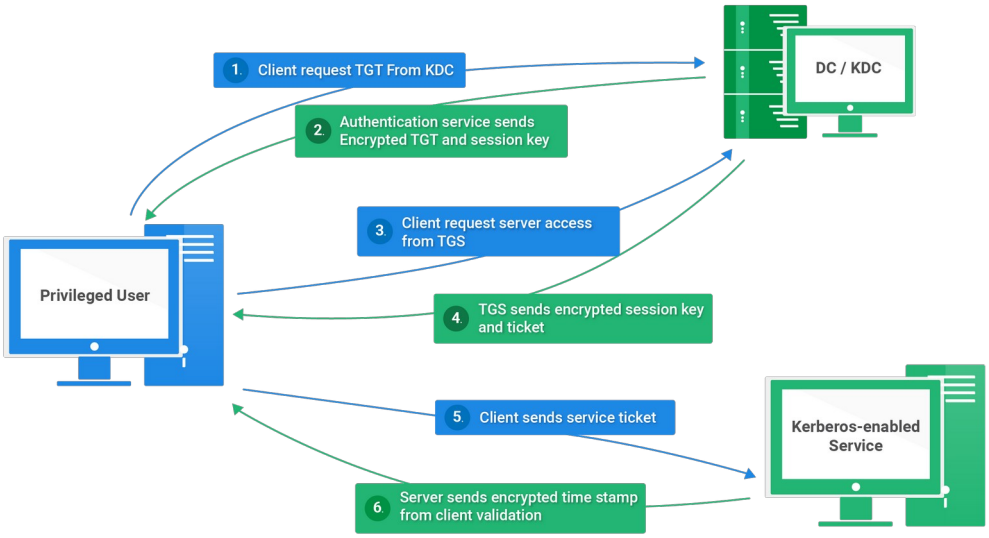
`sufficient` module succeeds and no `required` modules have failed, PAM immediately grants access without processing any further modules.

- **optional**:
    - Modules set as `optional` have the least impact on the outcome. They are typically used for non-critical tasks that do not directly influence the authentication outcome. The success or failure of an `optional` module is only important if it's the only module in the stack.

NSS (Name Service Switch):

- Works alongside PAM to retrieve account information from various sources, which can include local files like `/etc/passwd` or network-based sources such as LDAP.
- Allows system applications to retrieve user information transparently across different underlying technologies.

Integration with Kerberos for Enhanced Security



Kerberos Overview:

- Kerberos uses strong symmetric-key cryptography to authenticate users to network services, ensuring that passwords are never transmitted over the network.
- Provides Single Sign-On (SSO) capabilities, enabling users to authenticate once and gain access to multiple services without re-authenticating.

Key Concepts in Kerberos:

- **Realm:** A network segment that shares the same Kerberos database of users.
- **Principal:** A unique identity (user or service) in the Kerberos database.
- **Ticket:** A time-limited token that proves the identity of a user to a service.

Kerberos Authentication Process:

1. **Authentication**: Users authenticate to the Kerberos server and receive a Ticket Granting Ticket (TGT).
2. **Service Request**: Users request access to services using their TGT to obtain service-specific tickets.
3. **Service Access**: Service tickets allow users to access services securely and transparently.

Centralized Authentication Alternatives

Other Solutions:

- **FreeIPA**: Provides identity, policy, and audit management.
- **Samba 4**: Can act as an Active Directory compatible server.
- **Windows Active Directory**: Commonly used in environments with Windows servers and desktops.

Overview of LDAP, PAM, and Kerberos

LDAP (Lightweight Directory Access Protocol):

- **Use Cases**: LDAP is primarily used for directory services where there is a need to store and retrieve organizational data and resources in a structured directory. The protocol is used to manage user information, such as usernames, passwords, and other attributes like email addresses and departmental details.
- **Pros**:
    - **Scalability**: Efficiently handles a large number of read operations, making it ideal for large organizations.
    - **Flexibility**: Supports various authentication mechanisms and can integrate with multiple platforms.
    - **Centralized Management**: Centralizes user management, reducing redundancy and inconsistency.
- **Cons**:
    - **Complexity in Setup and Management**: Can be complex to set up and administer.
    - **Performance**: While optimized for reads, write operations can be relatively slower.

PAM (Pluggable Authentication Modules):

- **Use Cases**: PAM is used to integrate multiple authentication technologies into Unix-like operating systems without modifying existing applications. It provides a layer that enables the system and applications to authenticate users against different authentication backends.
- **Pros**:
    - **Modularity**: Allows system administrators to add or remove authentication methods without altering system services.
    - **Flexibility**: Supports a wide range of authentication schemes, which can be stacked in various configurations.

    - **Extensibility**: New authentication methods can be developed and plugged into the system seamlessly.
- **Cons**:
    - **Complex Configuration**: Can be complex to configure correctly, especially with multiple stacked modules.
    - **Security Risks**: Incorrect configurations can introduce security vulnerabilities.

Kerberos:

- **Use Cases**: Kerberos is used for secure user authentication in a client-server model. It is widely employed in secure environments where transmitting passwords or sensitive information over the network needs to be avoided.
- **Pros**:
    - **Security**: Provides strong cryptographic authentication, ensuring that passwords are not transmitted over the network.
    - **Single Sign-On (SSO)**: After initially authenticating, users can access multiple services without re-authenticating.
    - **Mitigates Replay Attacks**: Uses timestamps and session tickets to prevent replay attacks.
- **Cons**:
    - **Complexity**: The setup and maintenance of a Kerberos system can be complex.
    - **Dependency on Time**: Requires synchronized system clocks between the client and server.
    - **Limited Forgiveness**: Users can be locked out if their credentials are mishandled or if the system malfunctions.

How LDAP, PAM, and Kerberos Work Together

- **Integration for Comprehensive Solutions**: In many environments, these technologies are not used in isolation but are integrated to provide comprehensive security solutions. For instance, an organization might use LDAP to store user credentials and other details, Kerberos for authenticating these users securely, and PAM to integrate this authentication method transparently with various local services and applications.
- **LDAP and Kerberos**: LDAP can be used to store Kerberos data such as principals and their keys, making it easier to manage in large deployments.
- **PAM and LDAP**: PAM can be configured to use LDAP for user authentication, allowing applications that rely on PAM to authenticate users stored in an LDAP directory.
- **PAM and Kerberos**: Similarly, PAM can be configured to use Kerberos for performing the actual authentication step, thereby leveraging Kerberos' strong cryptographic security model.

7. Manage Permission, Ownership and Resources

In the complex landscape of modern computing, the concepts of authentication, authorization, and access control stand as fundamental pillars ensuring that systems remain both functional and secure. As networks grow and multiple machines interact, the challenges associated with managing local user accounts become increasingly apparent. Authentication is about verifying a user's identity through methods like usernames and passwords, ensuring they are who they claim to be. Once a user’s identity is authenticated, authorization plays its part by defining what the user can do within the system, such as determining which files they can access and the actions they can perform.

Authentication and Authorization

- **Authentication**: Verifies the identity of a user, typically through username and password. It ensures that users are who they claim to be.
- **Authorization**: Determines what an authenticated user is permitted to do within the system. Examples include:
    - Utilizing group permissions to grant access.
    - Distinguishing between regular user capabilities and those available to the root user.

Access Control

Operating systems like Linux utilize a Discretionary Access Control (DAC) security model, where the ownership of a resource, such as a file, grants the authority to dictate access permissions. This model is visually and conceptually illustrated through the DAC diagram, where the owner can set permissions that determine how other users interact with the resource. Permissions in Linux are categorized into read, write, and execute, and can be set for different groups of users: the owner, group members, and others.

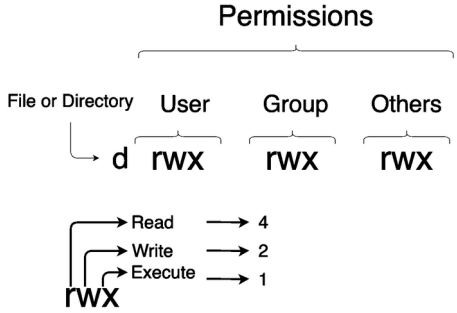
- **Discretionary Access Control (DAC)**:
    - Owners of resources (like files) decide who can access these resources.
    - Rights are granted by the resource owner, which reflects in the access settings.

Permissions

File Permissions:

- Control access through settings for read, write, and execute.
- Permissions apply to three different categories:
    - Owner
    - Group
    - Others (all users)

Permissions	Binary	Octal	Description
-	000	0	No permissions
–x	001	1	Execute-only permission
-w-	010	2	Write-only permission
-wx	011	3	Write and execute permissions
r–	100	4	Read-only permission
r-x	101	5	Read and execute permissions
rw-	110	6	Read and write permissions
rwx	111	7	Read, write, and execute permissions

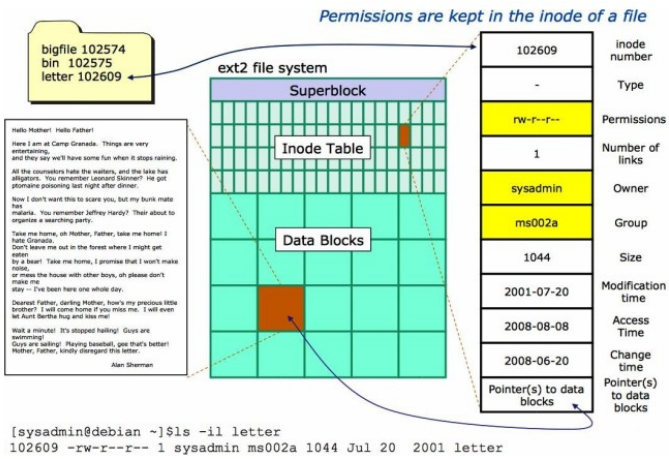


```
zaira@Zaira:~/freeCodeCamp$ ls -l
total 3856
-rw-r--r--    1 zaira zaira      89 Apr  5 20:46 CODE_OF_CONDUCT.md
-rw-r--r--    1 zaira zaira     210 Apr  5 20:46 CONTRIBUTING.md
-rw-r--r--    1 zaira zaira    1513 Apr  5 20:46 LICENSE.md
-rw-r--r--    1 zaira zaira   19933 Apr  5 20:46 README.md
drwxr-xr-x    4 zaira zaira    4096 Apr  6 22:45 api-server
-rw-r--r--    1 zaira zaira      67 Apr  5 20:46 babel.config.js
drwxr-xr-x   10 zaira zaira    4096 Apr  6 22:55 client
drwxr-xr-x    5 zaira zaira    4096 Apr  6 22:54 config
```

MODE OWNER GROUP SIZE MODIFICATION DATE FILE/FOLDER NAME

Inode

An inode (index node) is a data structure used to store the metadata of a file. Metadata includes information about a file's attributes such as its size, owner, group, permissions, and time stamps (like modification time, access time), as well as pointers to the disk blocks that store the file's actual data.



Key Aspects of the Inode:

1. **Identification**:
   - Each inode is uniquely identified by an inode number within the filesystem. This identifier is used internally by the system to access file metadata. In your displayed output, the inode number is `102609`.
2. **Permissions**:
   - File permissions are stored in the inode. These determine the actions that users can perform on the file, categorized by owner, group, and others. For example, `-rw-r--r--` indicates that the owner has read and write permissions, the group has read permissions, and others also have read permissions.
3. **Ownership**:
   - The inode contains information about the file's owner and group. This determines which user and group permissions apply to the file. In the example, `sysadmin` is the owner and `ms002a` is the group associated with the file.
4. **File Type**:

- Inodes store the type of the file (e.g., regular file, directory, link). This can be seen in the output where the file type is indicated as a regular file (`-` at the beginning of the permissions string).

5. **Links Count**:
    - This represents how many hard links point to the inode. A new file typically has a link count of `1`, indicating that there's one direct reference to it.
6. **File Size**:
    - The size of the file in bytes is stored in the inode, which helps the system determine how much data it needs to read or how much space to allocate when writing. In the example, the size is `1044` bytes.
7. **Pointers to Data Blocks**:
    - The inode does not contain the actual data of the file but points to the data blocks where the file’s data is stored on the disk. These pointers are crucial for the system to locate and interact with the file's contents.

8. **Time Stamps**:
    - Time stamps such as modification time, access time, and change time are also stored in the inode. These are useful for tracking when the file was last accessed, modified, or when its metadata was changed.

Special File Permissions (SUID, SGID, Sticky bit)

Special permissions such as Set User ID (SUID), Set Group ID (SGID), and sticky bits offer additional control. SUID allows a user to run a program with the permissions of the program’s owner, which can be crucial for tasks requiring elevated privileges, such as changing passwords. SGID, similar to SUID, applies this concept to group permissions and can ensure that files created within a directory inherit the group ID of the directory, not just the user who created them. The sticky bit is used to protect files within shared directories, where it restricts file deletion to the file's owner, the directory's owner, or the superuser.

- **Set User ID (SUID)**:
    - **Purpose**: Allows a user to run a program with the permissions of the program’s owner.
    - **Common Uses**: Crucial for tasks that require elevated privileges, such as managing passwords or modifying system files that require root access.
    - **Example**: The `/usr/bin/passwd` program typically has the SUID bit set so that regular users can change their passwords, which involves writing to the `/etc/shadow` file, normally only writable by the root user.
    - **Security Considerations**: If misused or incorrectly set, SUID can lead to significant security vulnerabilities, especially if executable files with SUID permissions are exploitable.

- **Set Group ID (SGID)**:
  - **Purpose**: Operates similarly to SUID but applies the group's permissions to the user who is running the executable.
  - **Common Uses**:
    - On executable files: Runs the executable with the permissions of the group owner, allowing users to perform tasks as a group that they individually might not have rights to perform.
    - On directories: Ensures files created within the directory inherit the directory's group ID, not the creating user's primary group ID.
  - **Example**: A shared directory where project files are stored could have SGID set so that new files created there automatically have the group ownership necessary for collaborative access.
  - **Security Considerations**: Like SUID, incorrect configuration or misuse of SGID can lead to unintended privilege escalation.

- **Sticky Bit**:
  - **Purpose**: Prevents users from deleting files they do not own within a shared directory.
  - **Common Uses**: Commonly set on world-writable directories like `/tmp` to ensure that while users can create files, they can only delete their own.
  - **Example**: If you set the sticky bit on `/tmp`, it prevents users from deleting or renaming files owned by other users, which can protect against malicious or accidental deletions.
  - **Security Considerations**: The sticky bit is primarily a safety feature and poses minimal security risks; however, it is crucial for maintaining the integrity of shared directories.

Working with SUID, SGID, and Sticky Bit

Permission	Numerical Value	Relative Value	On Files	On Directories
SUID	4	u+s	User executes file with permissions of file owner.	No meaning.
SGID	2	g+s	User executes file with permissions of group owner.	File created in directory gets the same group owner.
Sticky bit	1	+t	No meaning.	Users are prevented from deleting files from other users.

Examples:

Set SUID: `chmod 4755 file.txt`

Set SGID: `chmod 2755 file.txt`

Set sticky bit: `chmod 1755 /myDir` (set Sticky Bit to /myDir)

Commands for Managing Permissions

- `chown`: Change file owner.
- `chgrp`: Change group ownership of a file.
- `chmod`: Change file permissions.
    - Numeric mode (e.g., `chmod 640 file`)
    - Symbolic mode (e.g., `chmod u+rw file`)

Access Control Lists (ACLs)

In environments where traditional DAC mechanisms fall short, Access Control Lists (ACLs) provide a more flexible solution by allowing detailed access rights to be specified for additional users and groups. This is essential in complex systems where multiple entities require varied access levels to resources.

- **Enhanced Permissions Control**:
    - Allow for more granular permission settings beyond the owner-group-others model.
    - Enable setting permissions for multiple users and groups.
- **Commands**:
    - `setfacl`: Set specified ACLs on files/directories.
    - `getfacl`: Retrieve ACL information from files/directories.

Issues with Discretionary Access Control (DAC)

- **Limitations**:
    - Can't specify complex access scenarios easily.
    - No method to inherit rights directly through traditional permissions.
    - Not scalable. As the number of users and resources grows, managing DAC can become increasingly cumbersome.
    - Difficult enforcing company-wide policies.
    - Lack of audit trail. Many DAC systems do not provide comprehensive audit trails for changes in permissions or access patterns.
    - Users can grant access to any other user, which can lead to excessive permissions.

Mandatory Access Control (MAC)

In environments where traditional DAC mechanisms fall short, Access Control Lists (ACLs) provide a more flexible solution by allowing detailed access rights to be specified for additional users and groups. This is essential in complex systems where multiple entities require varied access levels to resources.

Linux also supports Mandatory Access Control (MAC) through implementations like SELinux and AppArmor, which restrict system resources based on policies set by an administrator, not by the

owning entity. This level of control is crucial for enforcing security policies that go beyond the capabilities of traditional DAC.

- Implemented via SELinux, AppArmor for enhanced security beyond DAC.
- Defines strict access rules managed by the system, not by the users, aligning with organizational security policies.

Default Permissions and Umask

- **Umask**:
    - Sets the default permission mask to define which permissions are removed when new files or directories are created.

Resource Management

Resource management is a crucial aspect of system administration that ensures optimal performance and stability of computer systems. Effective resource management prevents resource exhaustion, which can lead to system slowdowns or crashes, particularly in environments with high demands on computing power or multiple users. By managing resources like CPU time, RAM, disk space, disk I/O, and network bandwidth, administrators can allocate these assets appropriately among users and applications, ensuring that critical processes receive the priority they require. This not only maximizes efficiency but also maintains a fair usage policy, preventing any single process or user from monopolizing system resources.

For example, using tools like Disk Quota, `ulimit`, and Control Groups (`cgroups`), administrators can set hard and soft limits on the amount of resources a single user or process can consume. These tools help in maintaining system integrity and improving the overall user experience by balancing resource allocation in a way that supports the needs of diverse applications and services running on the system.

- **Resource Management Tools:**
    - Disk Quota: Manages disk space limits.
    - `ulimit`: Controls the usage of system resources.
    - Control Groups (`cgroups`): Manages and monitors system resources.

Administration with sudo

Sudo is a powerful means of access control. Sudo allows administrators to grant specific privileges to users without sharing the root password, thereby minimizing the risk of unauthorized system changes. It provides a detailed audit trail by logging all sudo commands, which is invaluable for troubleshooting and monitoring system activity. By configuring `sudoers` file, administrators can finely control who can run what commands on which machines, enhancing the security and manageability of multi-user systems. This granularity helps enforce the principle of least privilege, ensuring users have just enough access to perform their tasks without unnecessary exposure to sensitive system components or data.

- **Privilege Escalation**:

    - Allows specific users to execute commands with elevated privileges, mitigating the risks associated with full root access.
- **Configurable Access Control**:
    - `sudo` leverages the `/etc/sudoers` file to define which users or groups can execute particular commands on particular machines. This granularity ensures that permissions are tightly controlled.
- **Command Aliasing**:
    - Within the `sudoers` file, administrators can group related commands into aliases, simplifying management and enhancing readability.
- **Environment Control**:
    - `sudo` provides options to either preserve the user's environment when running a command or reset it to a clean, default set defined by the root environment. This flexibility can be critical for running scripts that require specific environment settings.
- **Audit and Accountability**:
    - Every command executed using `sudo` is logged. This logging includes information about who executed the command, what command was executed, and when it was executed. This is crucial for security, particularly in environments subject to compliance with audit requirements.
- **Security Features**:
    - Features like requiring a re-authentication after a set timeout period, logging failed `sudo` attempts, and the ability to restrict shell escapes. These features help to mitigate potential security risks associated with greater command access.
- **Lecture Capabilities**:
    - `sudo` can be set to lecture users about security practices the first time they use `sudo` or every time as a constant reminder of security responsibilities.
- **Command Integrity**:
    - By specifying full paths to commands in the `sudoers` file, administrators can prevent security breaches that might occur from malicious scripts or executables placed in user-controlled paths.

8. Network Attached Storage (NAS, NFS, Kerberos)

Network Attached Storage (NAS) provides an efficient solution to the limitations associated with managing local home directories across multiple clients and machines. As organizations grow, the need to access user files from various locations becomes critical, and managing these files directly on local machines can lead to inconsistencies in user data and challenges in file synchronization. NAS centralizes data storage on the network, allowing seamless file access and management across different devices without the complexities of local storage constraints. This setup not only simplifies data management but also enhances data security and accessibility, making it ideal for environments with extensive data sharing and collaboration needs.

NAS is a dedicated and specialized storage architecture that provides file-based data storage services to other devices on a network. Unlike Direct-Attached Storage (DAS), NAS connects directly to a network to serve data over network protocols.

Protocols: NAS typically uses Network File System (NFS) and Common Internet File System (CIFS) to provide network access to storage. NFS, developed by Sun Microsystems, is prevalent in UNIX/Linux environments, while CIFS is used extensively in Windows environments.

NFS (Network File System)

- Developed by Sun Microsystems, NFS has been a standard component of UNIX/Linux systems since the early 1980s.
- **Architecture**: NFS utilizes a client-server model where both the serving and receiving systems can be clients or servers.
- **Versions**:
    - **NFSv2**: Stateless and uses only UDP.
        - **Statelessness**: NFSv2 operates on a stateless protocol, meaning the server does not maintain any state information about the client sessions. Each request from the client must contain all the information necessary to fulfill the request.
        - **Protocol Support**: Initially, NFSv2 only supported the UDP protocol, which is connectionless and does not guarantee packet delivery, making it less reliable but faster in environments where reliability is less critical.
        - **Limitations**: This version is less secure because it relies heavily on the client's integrity, using the client-reported user identifiers for access control.
    - **NFSv3**: Adds support for TCP and remains stateless.
        - **Introduction of TCP**: NFSv3 added support for the TCP protocol, providing more reliable data transmission through connection-oriented service.
        - **Statelessness**: Like NFSv2, NFSv3 remains a stateless protocol, but with improved error handling and the ability to support larger files due to its support for 64-bit file sizes and offsets.
        - **Security and Performance**: It offers improved performance over NFSv2 and includes more robust error handling mechanisms, although it still inherits

some of the security weaknesses related to relying on client-side user information.

- **NFSv4**: Introduces stateful operations and mandates TCP for congestion control. It integrates well with Windows and Unix/Linux systems and includes built-in security enhancements through Kerberos.
    - **Stateful Operations**: NFSv4 introduced stateful operations, allowing the server to maintain client session states, which improves performance on operations requiring multiple requests and enhances security and network efficiency.
    - **Integrated Security**: It incorporates stronger security features natively, including the support for Kerberos authentication, which robustly handles user authentication and increases data security over the network.
    - **Unified Protocol**: NFSv4 consolidates file access, file locking, and mount protocols into a single protocol, reducing the complexity of firewall configurations and administration.
    - **Enhanced Interoperability**: Designed to work well in heterogeneous environments, NFSv4 supports Windows ACLs (Access Control Lists) and can more seamlessly integrate into Windows domains.

- **Security**:
    - **NFSv2 and NFSv3**: Security relies on IP-based access controls, which are less secure and suitable only within trusted networks.
    - **NFSv4**: Incorporates robust security features using the Kerberos protocol to authenticate users without transmitting passwords over the network.

NFS Security Problems

Problems with NFSv2 and NFSv3 Security:

1. **IP-based Authentication**:
    - Both NFSv2 and NFSv3 primarily use IP addresses for client authentication. This method is inherently insecure as IP addresses can be easily spoofed, allowing unauthorized users to access network resources by pretending to be trusted hosts.
2. **Trust-based Security**:
    - These versions operate under a trust-based model where the server trusts the client to authenticate locally and correctly report the credentials of the users accessing the NFS shares. This model does not verify the actual identity of the users, making it possible for users to access files they should not have access to by manipulating user IDs (UIDs) and group IDs (GIDs).
3. **Lack of Encryption**:
    - NFSv2 and NFSv3 do not support encryption, meaning that data transmitted over the network is susceptible to interception and eavesdropping. This lack of encryption is a critical flaw, especially when dealing with sensitive data across unsecured networks.

4. **Root Access Vulnerabilities**:
    - By default, NFSv2 and NFSv3 allow root users on the client machines unrestricted access to the exported filesystems, unless specifically restricted using the `root_squash` option which maps root access to a non-privileged user to mitigate this issue. However, this is more of a workaround than a solid security feature.

Security Enhancements in NFSv4:

1. **Mandatory Strong Authentication**:
    - NFSv4 requires strong authentication mechanisms, using Kerberos, a protocol that uses tickets to allow nodes communicating over a non-secure network to prove their identity to one another in a secure manner. Kerberos authentication ensures that user credentials are not only verified but also encrypted, preventing impersonation and credential theft.
2. **Integrated ACL Support**:
    - NFSv4 supports Windows-style ACLs in addition to traditional UNIX permissions. This feature allows more granular access control for files and directories, accommodating complex organizational policies and providing enhanced security over who can access specific data.
3. **Session Security**:
    - NFSv4 is a stateful protocol, which means it maintains session state information. This capability allows it to implement more sophisticated security measures, such as sequence number checks to prevent replay attacks where old communications are resent in an attempt to gain unauthorized access.
4. **All-in-one Protocol**:
    - By consolidating all NFS-related services into a single protocol that operates over one network port (2049), NFSv4 simplifies network management and reduces the number of entry points available to an attacker, thereby enhancing overall security.
5. **Encryption Support**:
    - NFSv4.1 and later versions include support for encryption, providing confidentiality in addition to authentication and integrity. This feature is crucial for securing sensitive data transfers over untrusted networks.

Kerberos with NFS

When integrated with NFS, Kerberos authenticates users but does not handle authorization, which is managed through file and directory permissions (ownership, ACLs).

Kerberos Authentication with NFS:

1. **Kerberos Tickets**:
    - When Kerberos is used with NFS, each client must obtain and present valid Kerberos credentials, typically in the form of a Kerberos ticket, to the NFS server. This ticket

is issued by the Kerberos Key Distribution Center (KDC) after the user's credentials have been successfully verified.

2. **Service Ticket Requirement**:
    - To access an NFS-exported filesystem, clients must present a service ticket specifically for the NFS service. This ticket proves that the user has been authenticated by the KDC and is authorized to access the service based on their credentials.
3. **Enhanced Security**:
    - The use of Kerberos ensures that the identity of all users accessing the NFS server is confirmed via a secure, encrypted authentication mechanism. This system helps prevent issues like identity spoofing and replay attacks, where an attacker could attempt to reuse an old ticket to gain unauthorized access.

Limitations in Authorization:

1. **Separation of Authentication and Authorization**:
    - While Kerberos effectively authenticates users, it does not manage what users can do once they have access. This separation is intentional to allow for flexible system management and ensure that authentication protocols do not become overly complex.
2. **Role of NFS Server**:
    - The NFS server uses the identity verified by Kerberos to consult existing access control mechanisms—such as UNIX file permissions, ownership, and Access Control Lists (ACLs)—to determine what operations a user can perform on a file or directory.
3. **Managing Permissions**:
    - Authorization decisions are based on the file and directory permissions set by the system or the data owner. These permissions define whether a user can read, write, or execute a file, or if further modifications can be made to file permissions.
4. **Kerberos Does Not Handle File Permissions**:
    - It is crucial to understand that while Kerberos can confirm who is trying to access a resource, it does not have a role in setting or enforcing the policies that dictate what authenticated users are allowed to do with that resource. These policies are handled entirely by the NFS server's configuration and the underlying operating system's file permissions system.

Components of NFS

- **mountd**: Manages mount requests from NFS clients.
- **nfsd**: Handles the actual data transfer.
- **lockd**: Manages file locking to prevent conflicts during file write operations.
- NFSv2 and NFSv3 requires to open multiple ports for the process. This is a challenge both for administrative and security purposes.

- NFSv4 simplifies administration by consolidating all traffic through port 2049, enhancing both security and management.

Server Configuration

- **/etc/exports**: This configuration file defines the directories available for network access and their permissions.
- **Syntax**: Defines the path, client, and options like `ro` (read-only), `rw` (read-write), and security controls such as `root_squash`.
    - **`/etc/exports:`** `/data/projects  192.168.50.12(rw,sync,no_root_squash)`
- **Common NFS export options**
    - **sync:**
        - **Description**: This option ensures that changes to the filesystem are written to disk before the server responds to the NFS request. It provides more data safety as the server waits for all file operations to be confirmed as written.
        - **Use Case**: Use `sync` to prevent data loss in case of a system crash or power failure. It is safer but may impact performance due to the wait involved in each operation.
    - **async**:
        - **Description**: Opposite of `sync`, this option allows the server to reply to requests before the changes made by the request have been written to the disk. It improves performance but at the risk of data loss if the server crashes.
        - **Use Case**: Good for environments where performance is critical and the risk of server crashes is either managed by other means (e.g., highly reliable hardware) or considered acceptable.
    - **root_squash**:
        - **Description**: Maps requests from the root user (UID 0) on the client to an anonymous UID and GID on the server. This prevents a root user on the client from having root-level access to the NFS mount, enhancing security.
        - **Use Case**: Essential for public or large multi-user environments where clients should not have administrative access to the NFS share.
    - **no_root_squash**:
        - **Description**: Does not map root user requests to anonymous IDs. This allows the root user on the client to have the same level of access as root on the server.
        - **Use Case**: Useful in trusted environments where administrative access is necessary across the server and clients, such as in managed IT infrastructures.
    - **subtree_check**:
        - **Description**: Ensures that the directories being accessed are part of the exported tree. This check can prevent access to files that are not properly exported but are part of the filesystem.

- **Use Case**: Increases security by ensuring that only explicitly shared parts of the filesystem are accessible. However, it can be disabled (using `no_subtree_check`) to improve performance if the entire filesystem is meant to be shared securely.

Client Setup

- **Mounting**: Clients mount NFS shares by specifying the server and the directory they wish to mount.
  - Create empty directory and mount the NFS directory to it
  - The mounted filesystem appears for end users as a part of the local filesystem
- **Syntax:** `mount -t nfs [hostname]:[remoteFilesystem] [mountPoint]`
  - **`/etc/fstab:`** `192.168.50.10:/data/projects /mnt/projects nfs defaults 0 0`
- **/etc/fstab**: Can be used to configure filesystems to mount automatically at boot.
- **Unmount:** Can be done with the `umount` command
  - `umount hostname:remoteFilesystem`
  - `umount mountpoint`
- **autofs**: Allows for dynamic mounting of filesystems when they are accessed, improving resource utilization and management.

Dynamically Mounting With autofs

Autofs is a service that provides automatic mounting of file systems, particularly useful for environments where file systems need to be accessed on-demand without permanently mounting them. This service can greatly enhance system performance and resource utilization by ensuring that file systems are mounted only when they are accessed and are unmounted after some period of inactivity.

How Autofs Works

- **Automount Daemon**: Autofs operates using the `automount` daemon, which listens for file system access in specific directories. When an access attempt is made, `automount` dynamically mounts the required file system.
- **Configuration**: The behavior of the `automount` daemon is configured through the `/etc/auto.master` file and other map files specified within `auto.master`.

Configuration Files

1. **/etc/auto.master**:
   - This is the main configuration file for autofs. It lists all the mount points managed by autofs, along with their corresponding map files and options.
   - **Syntax**: `mountpoint mapname [options]`
     - **mountpoint**: The directory where file systems will be automatically mounted (e.g., `/home`).
     - **mapname**: Path to the map file that contains details of what should be mounted and how.

    - **options**: Optional settings that can be applied to all entries in the map.
2. **Map Files**:
    - Map files detail the file systems to be mounted. These can be simple files like `/etc/auto.home` or more complex scripts or databases that provide dynamic mappings.
    - **Example Entry in a Map File**:
        - `* studssh.oslomet.no:/iu/cube/home/&`
        - This entry means that any directory accessed under the `/home` directory will be mounted from `studssh.oslomet.no:/iu/cube/home/[username]` where `[username]` corresponds to the accessed directory name.
3. **Examples:**
    - **Auto.master entry:**
        - `/home /etc/auto.home –timeout=300`
    - **Auto.home entry:**
        - `* -fstype=nfs,rw,soft,intr studssh.oslomet.no:/iu/cube/home/&`

Benefits of Using Autofs

- **Resource Efficiency**: Only mounts file systems when they are accessed, reducing resource consumption on the client machine.
- **Improved Performance**: Reduces the load time at boot since file systems are not mounted until needed.
- **Flexibility**: Particularly useful in large environments with many users or systems where constant access to all file systems is not necessary.
- **Scalability**: Handles changes in network file systems dynamically, allowing for scalability in large networked environments.

Autofs is particularly beneficial in network environments like educational institutions or large businesses, where user directories may be centrally hosted but only need to be accessed intermittently by the user or application processes. This setup ensures that the mounting process is seamless and transparent to the users while efficiently managing network resources.

9. Storage Area Network (iSCSI)

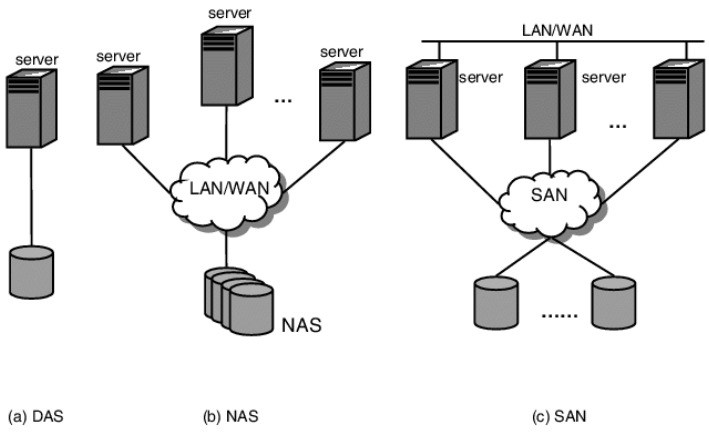
A **Storage Area Network (SAN)** is a dedicated high-speed network that provides access to consolidated block-level data storage. SAN is designed to handle large volumes of data transfer, enabling more efficient data storage, retrieval, and management than traditional networked storage solutions like Direct Attached Storage (DAS). It works on block-level data storage, unlike NAS which works on file level storage.

SAN offers significant advantages over traditional DAS by providing high-speed, flexible, and efficient access to consolidated storage resources. The ability to implement features like iSCSI and Multipath I/O further enhances its capability to support demanding data storage and retrieval requirements effectively.

Scenario and Problems with DAS

Direct Attached Storage (DAS): Initially, organizations might deploy servers with DAS, where each server has its own storage, such as two servers each with 2TB of storage.

Limitation: The primary limitation arises when one server (Server 1) runs out of storage while another (Server 2) might still have unused capacity. This isolated storage configuration prevents resource sharing between servers, leading to inefficient storage utilization.



Solutions and Limitations

- **Initial Solution**: One straightforward solution is to expand a server's storage capacity by adding external storage through interfaces like SATA/SCSI/SAS. However, this still does not address the inefficiency of unused storage in other servers.
- **Limitation with Initial Solution**: The inability to utilize unused storage on Server 2 for Server 1's needs highlights the inefficiency of DAS in environments requiring flexible resource management.

Transition to Network Storage

- **Advanced Solutions**: To overcome the limitations of DAS, organizations can deploy network storage solutions such as SAN or NAS, which allow storage resources to be shared across multiple servers.
    - **Network Attached Storage (NAS)**: Uses file-based storage protocols like NFS or CIFS.

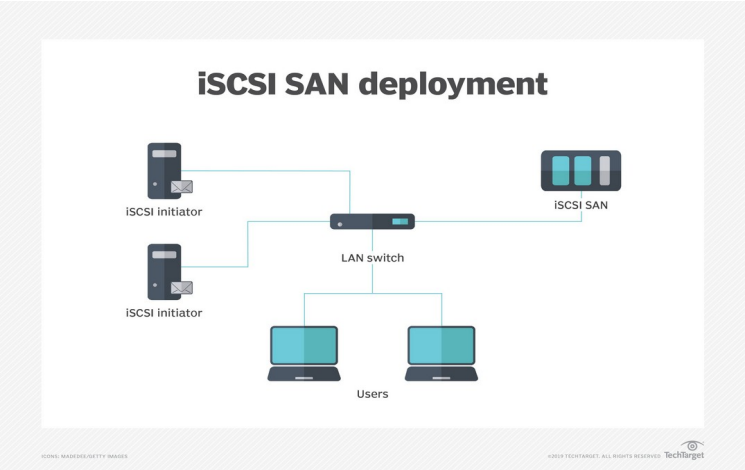
- **Storage Area Network (SAN)**: Uses block-level storage to provide more efficient and direct access to the disk storage.

SAN Configuration and Technologies

- **Core Concept**: SAN separates storage resources from network resources, linking storage directly to a dedicated network of storage devices via technologies like iSCSI, Fibre Channel (FC), and ATA over Ethernet (ATAoE).
    - **iSCSI (Internet Small Computer System Interface)**: A popular method that encapsulates SCSI commands into TCP/IP packets, allowing the SCSI protocol to be used over existing network infrastructures.

iSCSI Details

- ISCSI is a SAN technology which uses a TCP/IP network
- The iSCSI-protocol is an Internet Engineering Task Force (IETF) defined standard for IP storage
- The protocol sends SCSI commands in the TCP/IP packets
- Most of the commercial hardware vendors provide SAN storage solutions based on iSCSI
- Most OS also offer a software iSCSI solution
- **Architecture**: iSCSI operates on a client-server model:
    - **Initiator (Client)**: Sends SCSI commands to the server. Initiators can be software-based (running within the host OS) or hardware-based (using dedicated network cards).
    - **Target (Server)**: Receives SCSI commands and provides the storage resources.



- **Implementations**:
    - Various open-source and commercial options are available for setting up iSCSI targets and initiators on Linux and other operating systems.

Redundancy and Performance Enhancements

- **Multipath I/O**: This technique involves setting up multiple physical pathways between the client and storage server. Benefits include:
    - **Failover**: Provides redundancy, allowing data transfer to continue if one path fails due to hardware or connectivity issues.
    - **Increased Throughput**: Enhances performance by distributing the I/O load across multiple paths.

10.1. Dynamic Host Configuration Protocol (DHCP) and Static IP

In modern networks, managing IP addresses efficiently is crucial for ensuring reliable network communication. Two primary methods are employed to handle this task: Static IP configuration and Dynamic Host Configuration Protocol (DHCP).

Static IP Configuration

- **Manual Configuration**: Static IP addresses are manually assigned to a machine via a configuration file, GUI, or command line tools. This method allows precise control over network settings but can be cumbersome and error-prone in larger networks.
- **Tools for Configuration**:
    - **ifconfig**: Traditional tool for configuring network interfaces.
    - **ip**: A versatile tool that can manage routes, devices, policy routing, and tunnels.
    - **route**: Manages the kernel's IP routing tables.
    - **ethtool**: Queries and changes settings on an Ethernet device.
    - **mii-tool**: Manages media-independent interface status.

Challenges with Static IP Configuration

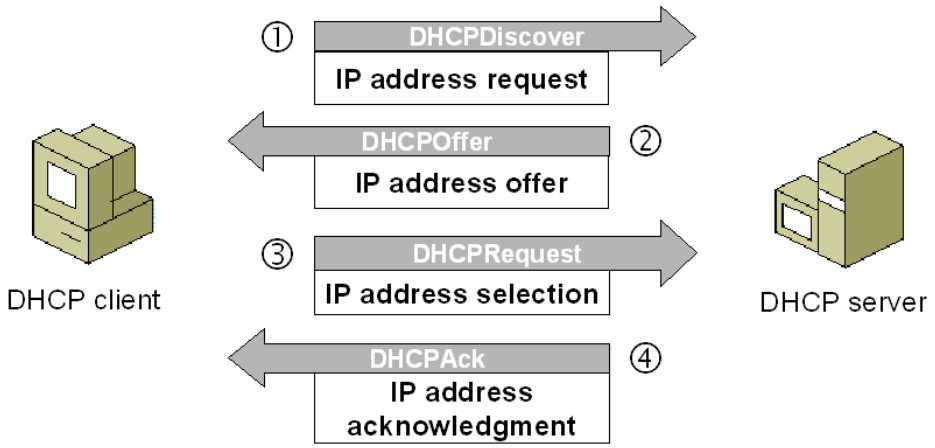
Managing IP addresses manually is not scalable for large networks due to the difficulty in tracking and updating configurations, leading to potential errors and operational inefficiencies.

Prior to DHCP, administrators had to enter all network information manually into a file. This does not scale and could lead to errors due to misconfigurations. Keeping the information updated and distributed was difficult to manage.

Dynamic Host Configuration Protocol (DHCP)

- **Automated Configuration**: DHCP automates the distribution of IP addresses and other network configuration details, making it scalable and error-resistant for managing large networks.
- **Operation**: DHCP servers assign IP addresses dynamically from a defined range and manage these assignments through leases. Clients contact the DHCP server to obtain or renew their IP configuration.
- **Benefits**:
    - Reduces administrative overhead.
    - Ensures that IP configurations are error-free and up-to-date.
    - Supports a wide range of configuration options as defined by IANA.

How DHCP Works



- **Process Overview**:
  - A client broadcasts a ***DISCOVER*** message to identify available DHCP servers.
  - A DHCP server responds with an ***OFFER*** message that includes an IP address and other network settings.
  - The client sends a ***REQUEST*** message to accept the offer and may use this message to renew or confirm an existing lease.
  - The server sends an ***ACK*** (acknowledgment) to confirm the lease or a ***NACK*** (negative acknowledgment) if there's an issue with the request.
  - Clients send a ***RELEASE*** message to terminate the lease voluntarily.

DHCP Messages Detailed

- **DISCOVER**: Finds DHCP servers on the network.
- **OFFER**: Proposes network settings to the client.
- **REQUEST**: Used by clients to accept an offer or renew a lease.
- **ACK/NACK**: Finalizes or denies the DHCP lease.
- **RELEASE**: Ends the DHCP lease.

ISC DHCP Implementation

- **Components**:
  - **Server**:
    - **Daemon**: `dhcpd`
    - **Config File**: `dhcpd.conf`
    - **Lease Database**: `dhcpd.leases`
  - **Client**:
    - **Agent**: `dhclient`
    - **Config File**: `dhclient.conf`
    - **Lease Database**: `dhclient.leases`

10.2. Domain Name Server (DNS)

DNS, the Domain Name System, functions like a phone book for the internet by translating human-friendly computer hostnames into IP addresses. For example, when you enter a web address such as `www.example.com`, DNS resolves this to the corresponding IP address `192.0.2.1`. This system is crucial for the functionality of the internet, making digital communications intuitive and efficient by allowing users to connect to websites using familiar domain names instead of numeric IP addresses.

Challenges and Solutions

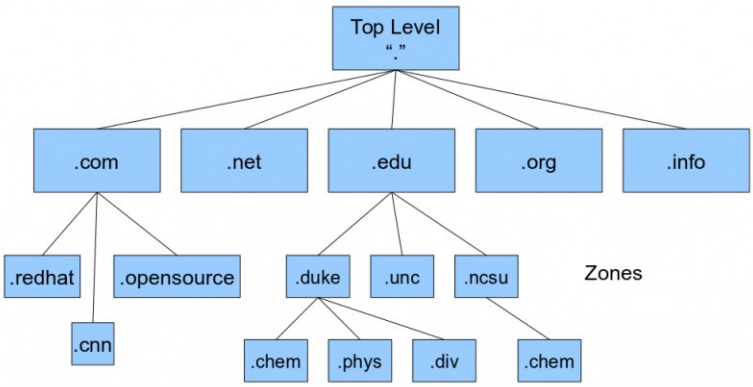
- **Hosts File**: Early methods of hostname resolution involved manually entering hostnames and IP addresses in a `hosts` file on each computer.
- **Scalability Issues**: As networks grew, maintaining a hosts file on each computer became untenable.
- **DNS as a Solution**: DNS automates the mapping of hostnames to IP addresses, eliminating the need for manual updates and greatly improving scalability and management.

DNS Configuration Files

- **nsswitch.conf**: Specifies how and where services should search for names service information, including DNS.
- **resolv.conf**: Contains information about DNS servers and is used for domain name lookups.

How DNS Works

- **Resolver and DNS Server Interaction**: If a hostname is not in the local hosts file, the resolver (DNS client) queries a DNS server to get the corresponding IP address.
- **Hierarchical Structure**: DNS is structured hierarchically, including root servers, TLD servers, and domain-specific servers, each responsible for a specific portion of the DNS namespace. Root and top-level domains are administred by ICANN. Below top-level domain, administration of name space is delegated to organizations. Each organization can delegate further.



- **DNS Queries**: Can be either recursive, where the local DNS server queries other DNS servers on behalf of the client, or iterative, where the DNS server refers the client to another server.

DNS Queries

- **Recursive Queries**: The client expects a direct answer from the DNS server. If the server doesn't know the answer, it will query other servers.
- **Iterative Queries**: The DNS server provides the best answer it can and may refer the resolver to another DNS server closer to the domain in question.

DNS Server and Records

- **DNS Servers**: Manage the mapping of domain names to IP addresses and handle queries about domain names within their zones.
    - The organization which owns a domain is responsible for running a DNS server that can provide mappings between machine names within its domain to correct IP addresses. These mappings are in the zone files.
    - The organization must have a primary server for the domain, and one or more secondary servers containing a copy of the zone files.
- **Resource Records in DNS**: These records contain information about domains and their associated services, such as NS (name server), A (address record), and MX (mail exchange).
- **Zone Files**: Contain resource records for all the domains within a zone and are used by the DNS server to resolve queries.

DNS Tools

- **Tools**: Tools like `nslookup`, `host`, and `dig` are used for querying DNS servers to understand and diagnose DNS responses.
- **ISC BIND**: The most common DNS server software, BIND (Berkeley Internet Name Domain), is responsible for implementing DNS protocols and providing DNS services.

11. Email System (SMTP, IMAP, Webmail)

Email remains one of the most ubiquitous applications on the Internet, with billions of messages sent daily. It facilitates communication across vast distances instantly and is pivotal in both business and personal contexts. To manage this, various systems and protocols have been established to ensure emails are sent, received, and stored efficiently.

Email System Components

- **Mail User Agent (MUA)**: This is the application used to write, send, and read emails. Examples include Thunderbird, Outlook, and Apple Mail. The MUA interacts directly with a Mail Transport Agent (MTA).
- **Mail Transport Agent (MTA)**: Responsible for sending email from the sender to the recipient. MTAs use the Simple Mail Transfer Protocol (SMTP) to exchange email across networks.

Email Transmission: SMTP

- **SMTP Details**: SMTP is the standard protocol for sending email messages between servers. It handles the "envelope" and "body" of emails, ensuring messages are correctly routed and delivered.
- **SMTP Server**: Acts as a post office, sending and receiving emails. Popular open-source SMTP servers include Postfix, Sendmail, and Exim4.

Email Retrieval: IMAP and POP3

- **IMAP (Internet Message Access Protocol)** and **POP3 (Post Office Protocol)** are the main protocols used for retrieving email from an MTA. IMAP is more feature-rich, allowing for better management of messages across multiple devices.
- **Email Access**: Users can access their email directly through the MTA or use IMAP/POP3 to retrieve messages to their MUA.

Email Storage: Mbox vs Maildir

- **Mbox**: Stores all emails in a single file. This method is simpler but can become inefficient with large volumes of mail.
- **Maildir**: Stores each email in a separate file within a directory structure, improving performance and reliability.

Handling Multiple Domains

- **Single MTA, Multiple Domains**: A single MTA can manage email for multiple domains, which can be configured in various ways such as shared or separate accounts.
- **Virtual Hosting**: Allows for the management of emails across multiple domains without requiring separate system accounts for each domain.

Security and Management

- **SMTP Authentication (SMTP-AUTH)**: Enhances security by requiring email clients to authenticate before sending emails, helping to secure email transactions.
- **Anti-Spam and Anti-Virus**: Essential tools for any email system to protect users from unwanted emails and potential security threats.

Webmail Systems

- **Functionality**: Webmail allows users to access their email through a web browser, providing the flexibility to manage email from any device connected to the internet.
- **Popular Webmail Solutions**: Include both open-source projects like SquirrelMail and Roundcube, and proprietary services like Gmail and Outlook Web App.

Important email port numbers:

Protocol	**Use**	**Plain/Encrypt**	**Plain only**	**Encrypt only**
SMTP	Outgoing	25	X	*465
MSA	Outgoing	587	X	X
POP3	Incoming	110	X	995
IMAP4	Incoming	143	X	993
HTTP	Webmail	X	80	443

* Revoked/unofficial

12. Miscellanous

Here are some notes of different important topics not covered explicitly in our lectures.

Default ports

Short list:

Port 20 & 21: FTP for file transfer.

Port 22: SSH for secure remote administration.

Port 23: Telnet for insecure remote administration.

Port 25: SMTP for email transmission.

Port 53: DNS for domain name mapping.

Port 80: HTTP for unsecured web traffic.

Port 110: POP3 for receiving email.

Port 143: IMAP for email retrieval.

Port 443: HTTPS for secure web traffic.

Port 465 & 587: SMTPS/SMTP for secure email transmission.

Port 993: IMAPS for secure email retrieval.

Port 995: POP3S for secure email receiving.

Port 3306: MySQL for database access.

Port 3389: RDP for remote desktop protocol.

Port 5432: PostgreSQL for database access.

Command to list ports: `grep <port> /etc/services`

Detailed list of common ports related to sysadmin tasks:

SSH (Secure Shell) - Port 22 Used for secure remote access to servers, allowing encrypted communications between hosts.	**HTTP (Hypertext Transfer Protocol) - Port 80** The fundamental port for web traffic, used by web browsers to access websites without encryption.
HTTPS (HTTP Secure) - Port 443 Secure version of HTTP, encrypting communications to ensure privacy and security over the web, used extensively for transactions and all secure web applications.	**FTP (File Transfer Protocol) - Port 21** Used for the transfer of computer files between a client and server on a computer network. FTP has a secure variant, FTPS, which also uses port 990 for implicit connections.

SMTP (Simple Mail Transfer Protocol) - Port 25
Primary protocol for sending email across the Internet. Port 587 is also commonly used for SMTP message submission.

DHCP (Dynamic Host Configuration Protocol) - Ports 67 and 68
Assigns IP addresses and other network configuration parameters dynamically to devices on a network. Port 67 is used by the server, and port 68 is used by the client.

SNMP (Simple Network Management Protocol) - Ports 161 and 162
Used for monitoring, configuring, and managing network devices. Port 161 is used for SNMP commands, while port 162 is designated for SNMP traps.

SMB (Server Message Block) - Port 445
Provides shared access to files, printers, and serial ports and miscellaneous communications between nodes on a network.

MySQL - Port 3306
Default port used by the MySQL database management system for operating database functions over the network.

Elasticsearch - Port 9200
Utilized by Elasticsearch for HTTP communication in distributed search and data analytics engine.

iSCSI / iqn - Port 3260
Internet Small Computer Systems Interface (iSCSI) uses this port to enable data transfers over intranets and to manage storage over long distances.

DNS (Domain Name System) - Port 53
Resolves human-readable domain names to IP addresses, allowing browsers and other services to connect to servers.

NTP (Network Time Protocol) - Port 123
Synchronizes clocks of networked devices to within a few milliseconds of coordinated universal time (UTC).

LDAP (Lightweight Directory Access Protocol) - Port 389
Manages and accesses distributed directory information services. Port 636 is used when LDAP is run over SSL (LDAPS).

RDP (Remote Desktop Protocol) - Port 3389
Enables remote desktop connections, allowing users to control systems remotely.

PostgreSQL - Port 5432
Similar to MySQL, this port is used by the PostgreSQL database management system to listen for database queries from clients.

Memcached - Port 11211
Employed by Memcached, a general-purpose distributed memory caching system often used to speed up dynamic web applications.

Telnet - Port 23
Used for unencrypted communication with remote servers, primarily for accessing and managing devices. Not recommended for secure communications due to its lack of encryption.

IMAP (Internet Message Access Protocol) - Port 143 Used for retrieving emails from a mail server, allowing multiple clients to manage the same mailbox. Port 993 is used for IMAP over SSL (IMAPS).	**POP3 (Post Office Protocol version 3) - Port 110** Another protocol for retrieving email from a server, simpler than IMAP but with less functionality. Port 995 is used for POP3 over SSL (POP3S).
SFTP (SSH File Transfer Protocol) - Port 22 Used for secure file transfer operations over the SSH protocol. It provides file access, file transfer, and file management functionalities.	**SCP (Secure Copy Protocol) - Port 22** Allows files to be securely transferred between hosts on a network. Uses SSH for data transfer and authentication, ensuring the confidentiality and integrity of the data.
TFTP (Trivial File Transfer Protocol) - Port 69 A simple, lightweight file transfer protocol with no authentication, commonly used for booting from a network (e.g., PXE boot) or transferring configuration files to network devices.	**LDAP over SSL (LDAPS) - Port 636** Secure version of LDAP, which is necessary when handling sensitive data across networks.
Kerberos - Port 88 Used for network authentication protocols designed to provide strong authentication for client/server applications by using secret-key cryptography.	**OpenVPN - Port 1194** Default port for OpenVPN applications. OpenVPN is an open-source software application that implements virtual private network techniques.
Microsoft SQL Server - Port 1433 Used by Microsoft SQL Server for database management connections. A crucial port for environments running Microsoft SQL databases.	**MongoDB - Port 27017** Default port used by MongoDB, a NoSQL database, to establish connections.
SMTPS (Secure SMTP) - Port 465 Designated for SMTP connections encrypted from the start, as opposed to upgrading from an insecure connection.	**RSYNC - Port 873** Used for fast, incremental file transfer, both locally and remotely. Useful for backups and mirroring.
PPTP (Point-to-Point Tunneling Protocol) - Port 1723 Used for creating VPN connections, often encapsulated over GRE (Generic Routing Encapsulation).	**BGP (Border Gateway Protocol) - Port 179** Essential for exchanging routing information between gateway hosts in a network of autonomous systems on the internet.

Useful commands

List some usefull commands, sorted by topic

Network Management

- **`ip`**: Show / manipulate routing, devices, policy routing, and tunnels.
- **`netstat`**: Displays network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.
- **`ping`**: Check the network connection to a server.
- **`traceroute`**: Trace the route packets take to a network host.
- **`ifconfig`**: Configure the kernel-resident network interfaces.
- **`route`**: Manipulate the kernel's IP routing tables.
- **`ethtool`**: Query or control network driver and hardware settings.
- **`mii-tool`**: Deprecated tool similar to ethtool, used for checking or setting the status of a network interface's Ethernet connection.
- **`dig:`** A network administration command-line tool for querying Domain Name Systems (DNS) to obtain domain name or IP address mapping information.

User and group management

- **`useradd`**, **`usermod`**, **`userdel`**: Commands for creating, modifying, and deleting user accounts.
- **`groupadd`**, **`groupdel`**, **`groupmod`**: Manage group accounts.
- **`passwd`**: Command to change user's password.
- **`id`**: Print real and effective user and group IDs.
- **`edquota`**: Edit user quotas for file systems.
- **`repquota`**: Summarize quotas for a file system.
- **`quotacheck`**: Scan a file system for disk usage, creating, checking, and repairing quota files.

File and Directory Operations

- **`ls`**, **`cp`**, **`mv`**, **`mkdir`**, **`rm`**: Basic file management commands.
- **`chmod`**, **`chown`**, **`chgrp`**: Modify file permissions and ownership.
- **`tar -xvzf, gzip, bzip2`**: Tools for compressing and packaging files.

Software Installation and Management

- **`apt`**, **`apt-get, aptitude`**: Commands for managing packages in various Linux distributions.
- **`dpkg`**: Debian package manager for installing, creating, and managing Debian packages.

- **`Make`**: Automates the compilation of programs and libraries from source code using a file called a Makefile to define build targets and dependencies
- **`checkinstall`**: Create a package from installed source.

System Boot and Service Management

- **`systemctl`**: Control the systemd system and service manager.
- **`service`**: Run a System V init script.
- **`journalctl`**: Query contents of the systemd journal.

Disk Management, file systems and storage

- **`fdisk`**, **`parted`**: Disk partitioning tools.
- **`mkfs`**, **`fsck`**: Create and check file systems.
- **`mount/umount`**: Mount and unmount file systems.
- **`df`**: Report file system disk space usage.
- **`du`**: Estimate file space usage.
- **`exportfs`**: Manage exported NFS and NCP (NetWare Core Protocol) file systems.
- **`numfmt`**: Convert numbers from/to human-readable strings.
- **`pvcreate, vgcreate, lvcreate`**: LVM commands for creating physical volumes, volume groups, and logical volumes respectively .
- **`pvdisplay, vgdisplay, lvdisplay`**: LVM commands for displaying physical volumes, volume groups, and logical volumes respectively .

Security and Firewall Management

- **`iptables`**, **`firewalld`**: Commands to manage network firewalls.
- **`openssl`**: Toolkit for SSL/TLS certificates.

LDAP Directory Services

- **`ldapsearch`**: Search LDAP directory entries.
- **`ldapadd`**: Add entries to an LDAP directory.
- **`ldapadduser`**: Add a user to an LDAP directory.
- **`ldapaddgroup`**: Add a group to an LDAP directory.
- **`ldapmodifyuser`**: Modify a user entry in an LDAP directory.

Security and Access Management

- **`getfacl`**: Display file access control lists.
- **`setfacl`**: Set file access control lists.
- **`slappasswd`**: Create an LDAP password for storing in slapd.conf.
- **`chage`**: Change user password expiry information.
- **`chpasswd`**: Update passwords in batch mode.

- **`gpasswd`**: Administer /etc/group and /etc/gshadow.
- **`getent`**: Get entries from administrative databases.

Email Management

- **`swaks`**: A testing tool for SMTP servers.
- **`postconf`**: Query or set Postfix configuration options.
- **`getmail`**: Fetch mail from remote servers to your local machine.

Storage and Virtualization

- **`iscsiadm`**: Tool to manage iSCSI sessions and targets.
- **`targetcli`**: Administers iSCSI targets via a command-line interface.
- **`lxc`**: Command-line interface for managing LXC containers.
- **`lvextend`**: Extends the size of a logical volume.
- **`mdadm`**: Manage MD (multiple disk) devices aka Linux Software RAID.
- **`blkid`**: Locate/print block device attributes.

Configuration Files and Locations

Common configuration file paths and their purposes.

User and Group Management

- **`/etc/passwd`** and **`/etc/shadow`**: User account information.
- **`/etc/group`**: Defines groups, listing users who are members of each group.
- **`/etc/ldap.conf:`** Global LDAP configuration file to configure client applications like `ldapsearch`.
- **`/etc/ldapscripts/ldapscripts.conf:`** Configuration for LDAP scripts used for managing users and groups in LDAP.

Network Configuration and Services

- **`/etc/hosts`**: Local host to IP mappings.
- **`/etc/nsswitch.conf`**: Service configuration for the Name Service Switch.
- **`/etc/resolv.conf`**: Resolver configuration file for DNS.
- **`/etc/network/interfaces:`** Configuration files for network interfaces
- **`/etc/ssh/sshd_config`**: Configuration for SSH daemon.
- **`/etc/dhcp/dhcpd.conf`**: Configure DHCP server.
- **`/etc/default/isc-dhcp-server:`** Specifies defaults for the ISC DHCP server, typically network interfaces to listen on.
- **`/etc/iscsi/iscsid.conf:`** Configuration for iSCSI daemon settings.
- **`/etc/iscsi/initiatorname.iscsi:`** Contains the unique initiator name for an iSCSI network boot.

Web and Mail Services

- **`/etc/postfix/main.cf:`** Main configuration for the Postfix mail server.
- **`/etc/postfix/master.cf:`** Specifies the behavior of different Postfix services.
- **`/etc/httpd/conf/httpd.conf`**: Configuration for Apache HTTP server.

DNS Configuration

- **`/etc/bind/named.conf.local:`** Specifies DNS zones for BIND.
- **`/etc/bind/named.conf.options:`** Sets global options for BIND DNS server.

File Systems and Storage

- **`/etc/fstab`**: Static file system information for mounting file systems.
- **`/etc/exports:`** Defines which file systems are exported to remote hosts and the access options.
- **`/etc/mdadm/mdadm.conf:`** Configuration for software RAID using `mdadm`.
- **`/etc/mtab:`** Currently mounted file systems.
- **`/proc/mounts:`** Shows all mounted file systems in real time, similar to `/etc/mtab`.

Automount and Remote File Systems

- **`/etc/auto.home`:** Map file used by `autofs` to automatically mount home directories.
- **`/etc/auto.master`:** Master map file for the automounter, `autofs`, specifying mount points.

System Information and Status

- **`/proc/mdstat`:** Virtual file providing current status of RAID disks.
- **`/proc/mounts`:** Shows all mounted file systems in real time, similar to `/etc/mtab`.